C / C++ / Java / Python / Go 语言与开发面试题汇总（附答案）

> 整理日期：2026-10-17 规模：C 110 题、C++ 117 题、Java 115 题、Python 108 题、Go 110 题，共 560 题。
>
> 来源说明：题目与答案是我根据常见面试考点和自己的知识整理的，不是从某个网站逐题摘录的。这次会话的出口策略阻止了对多数面经网站的抓取，所以没有逐题引用的外部来源。
>
> 使用提示：
>
> - 与语言版本相关的内容（Java 21/25、Python 3.13/3.14、Go 1.22–1.25、C++20/23、C23）变化较快，答案里已尽量标注版本，面试前请以官方文档为准。
> - 答案以"面试能讲清楚"为目标，写得简洁，需要深挖的话题（JVM 调优、Go 调度器、C++ 内存模型等）建议再读专门资料。
> - 每种语言的题号独立编号，按主题分节。

目录

- 第一部分：C 语言（110 题）
- 第二部分：C++（117 题）
- 第三部分：Java（115 题）
- 第四部分：Python（108 题）
- 第五部分：Go（110 题）

第一部分：C 语言

1.1 基础与类型系统

Q1. int、long、long long 各多大？

标准只保证 `char` ≥8 位，`short` / `int` ≥16 位，`long` ≥32 位，`long long` ≥64 位，且 `sizeof(short)≤sizeof(int)≤sizeof(long)≤sizeof(long long)`。常见 LP64（Linux/macOS 64 位）：int 4 字节、long 8 字节；LLP64（64 位 Windows）：long 4 字节、long long 8 字节。需要确定宽度时用 `<stdint.h>` 的 `int32_t`、`uint64_t` 等。

Q2. sizeof 与 strlen 有什么区别？

`sizeof` 是编译期运算符（VLA 除外），返回类型/对象占用字节数，包含结尾 `\0`；`strlen` 是运行期函数，遍历到 `\0` 为止，不含 `\0`。`char s[] = "abc"`：`sizeof(s)==4`，`strlen(s)==3`。数组作为函数参数时退化为指针，`sizeof` 得到指针大小。

Q3. 有符号整数溢出和无符号整数溢出有何不同？

无符号溢出是良定义的，按 2^N 取模回绕；有符号溢出是未定义行为，编译器可据此做优化（如假设 `x+1>x` 恒真）。检测溢出可用 `__builtin_add_overflow`（GCC/Clang），C23 提供 `<stdckdint.h>`。

Q4. 整型提升与通常算术转换是什么？`-1 < 1u` 结果如何？

小于 int 的类型运算前先提升为 int；不同类型运算时按"通常算术转换"转成同一类型。有符号与同级别无符号混合时，有符号转为无符号，所以 `-1` 变成 `UINT_MAX`，`-1 < 1u` 为 0（假）。这是经典陷阱，比较循环下标与 `size_t` 时尤其常见。

Q5. char 是有符号还是无符号？

实现定义：x86 的 GCC 上是有符号，ARM 的 Linux 上常为无符号。需要明确语义时用 `signed char` / `unsigned char`；用 `isalpha` 等函数时要把 char 先转为 `unsigned char`。

Q6. 为什么要用 stdint.h 里的固定宽度类型？

可移植地表达位宽：`int8_t` … `int64_t`、`uint*_t`，还有 `int_least*_t`、`int_fast*_t`、`intptr_t`、`size_t`、`ptrdiff_t`。协议解析、序列化、位操作、寄存器映射等场景必须使用固定宽度。

Q7. const 与指针的几种组合怎么区分？

从右向左读：`const int *p` 指向 const int（不能通过 p 改值，p 可改）；`int *const p`，p 本身是 const（不能改指向）；`const int *const p` 两者都不可改。`const` 只是编译期约束，通过强转修改真正的 const 对象是未定义行为。

Q8. volatile 的作用是什么？能保证线程安全吗？

告诉编译器该变量可能被外部（硬件寄存器、信号处理函数、setjmp 之后）修改，每次访问都必须真实读写，不能缓存到寄存器或消除。它不提供原子性，也不提供内存序，多线程同步应使用 C11 `_Atomic` 或互斥锁。

Q9. static 有哪些用法？

1. 函数内的 static 局部变量：存储期为整个程序，只初始化一次。2. 文件作用域的 static 变量/函数：内部链接，仅本翻译单元可见。3. 参数数组声明 `int a[static 10]`（C99）：表示至少 10 个元素。

Q10. extern 的作用？

声明一个在别处定义的对象或函数（外部链接），不分配存储。头文件里放 `extern int g;`，在一个 .c 文件里定义 `int g;`。函数声明默认就是 extern。

Q11. register 和 restrict 是什么？

`register` 是对编译器的存储提示（现代编译器基本忽略，且不能取地址）。`restrict`（C99）是程序员向编译器保证：在该指针的生命周期内，通过它访问的对象不会被其他指针访问，让编译器敢于向量化/重排。违反承诺是未定义行为，`memcpy` 的参数就带 restrict，所以重叠拷贝要用 `memmove`。

Q12. C 的存储类别与存储期有哪些？

存储类别说明符：`auto`、`register`、`static`、`extern`、`_Thread_local`（C23 为 `thread_local`）。存储期：自动、静态、线程、动态（malloc）。

Q13. 位域（bit-field）有什么注意点？

`struct { unsigned a:3; unsigned b:5; }` 可节省空间，但位序、对齐、跨存储单元行为都是实现定义，不能取地址，不适合跨平台协议，推荐用掩码+移位显式处理。

Q14. 如何判断机器大小端？

```
uint32_t x = 1;
int little = *(unsigned char*)&x == 1;   // 通过 char 访问是允许的
```

网络字节序为大端，用 `htonl/ntohl` 转换。

Q15. 为什么不能用 == 比较浮点数？

浮点有舍入误差（`0.1+0.2 != 0.3`）。应比较差值是否小于容差：`fabs(a-b) < eps`，或用相对误差/ULP 比较。NaN 与任何值都不相等，包括自己。

Q16. 什么是序列点（sequence point）？`i = i++ + ++i` 会怎样？

同一对象在两个序列点之间被修改多次，或修改同时又读取（非用于计算新值），是未定义行为。C11 改用"sequenced before"表述，结论相同。函数参数求值顺序、`+` 两侧求值顺序都是未指定的，只有 `&&`、`||`、`?:`、`,` 和完整表达式末尾有明确顺序。

Q17. 宏和 inline 函数怎么选？

宏是文本替换，无类型检查、可能重复求值（`MAX(a++,b)`）、难调试；inline 函数有类型检查和作用域，编译器通常能同等优化。优先 `static inline`，宏用于条件编译、字符串化、泛型小技巧（C11 `_Generic`）。

Q18. enum 与 #define 常量比较？

enum 有类型意识、调试器可见、自动递增；但 C 里枚举常量类型是 int（C23 允许指定底层类型）。`#define` 无作用域、无类型。整数常量集合用 enum，也可用 `static const`，但它在 C 里不是常量表达式（不能用作数组长度，VLA 除外）。

Q19. C11 的 _Generic 是做什么的？

编译期按表达式类型选择分支，实现简单的泛型/重载：

```
#define abs_any(x) _Generic((x), int: abs, long: labs, double: fabs)
(x)
```

Q20. C99 / C11 / C17 / C23 分别引入了什么重要特性？

C99：`//` 注释、`inline`、VLA、`restrict`、复合字面量、指定初始化器、`stdint.h`、`bool`。C11：`_Atomic`、`threads.h`、`_Generic`、`_Static_assert`、对齐控制、匿名结构/联合。C17：主要是缺陷修复。C23：`nullptr`、`constexpr`、`typeof`、`bool/true/false` 成为关键字、`#embed`、`[[attributes]]`、`<stdckdint.h>`、二进制字面量 `0b`。

1.2 指针与数组

Q21. 指针和数组有什么区别？

数组是一块连续的、大小固定的对象，`sizeof(arr)` 是整块大小，不能赋值；指针是存放地址的变量，`sizeof` 是指针大小。数组名在大多数表达式中会退化为指向首元素的指针，例外：`sizeof`、`&`、字符串字面量初始化数组。`a[i]` 等价于 `*(a+i)`，所以 `i[a]` 也合法。

Q22. 数组作为函数参数时会发生什么？

`void f(int a[10])` 等价于 `void f(int *a)`，长度信息丢失，函数内 `sizeof(a)` 是指针大小。需要显式传长度，或用 `int (*p)[N]` 指向数组的指针，或把数组包在结构体里传值。

Q23. 指针数组和数组指针的区别？

`int *a[10]`：数组，10 个元素，每个是 int*；`int (*p)[10]`：指针，指向含 10 个 int 的数组。口诀：运算符优先级中 `[]` 高于 `*`，括号改变结合。

Q24. 如何声明与使用函数指针？

`int (*fp)(int, int) = add;`，调用 `fp(1,2)` 或 `(*fp)(1,2)`。常用 `typedef int (*binop)(int,int);` 简化，用于回调、状态机跳转表、qsort 比较器。

Q25. void* 是什么？

通用指针，可与任意对象指针互转，不能直接解引用或做算术（GNU 扩展允许）。`malloc`、`qsort`、`memcpy` 都使用它。在 C 中无需强转，在 C++ 中需要。

Q26. 野指针和悬空指针是什么？如何避免？

野指针：未初始化的指针。悬空指针：指向的内存已被释放或离开作用域。避免：声明时初始化为 NULL、`free` 后置 NULL、不返回局部变量地址、用 ASan 检测。

Q27. NULL、0、nullptr 的关系？

NULL 通常是 `((void*)0)` 或 `0`；空指针在语义上与"地址全 0"不一定一致，但程序里用 0/NULL 初始化得到的就是空指针。C23 引入 `nullptr`，类型为 `nullptr_t`。

Q28. 指针算术的规则是什么？

`p + n` 前进 `n * sizeof(*p)` 字节；两个指向同一数组的指针相减得到 `ptrdiff_t`；只在同一数组（含末尾后一位）内做算术/比较才是良定义的。

Q29. 二级指针有什么用途？

1. 在函数里修改调用者的指针（如 `void alloc(int **p)`）；2. 指针数组/字符串数组（`char **argv`）；3. 动态二维数组；4. 链表头指针

的修改（删除头结点时免去特判）。

Q30. 二维数组如何传给函数？

列数必须已知：`void f(int a[][4], int rows)` 或 `int (*a)[4]`；C99 可用 VLA 参数 `void f(int r, int c, int a[r][c])`。若用 `int**` 则需要的是指针数组，两者内存布局不同，不能互换。

Q31. 什么是严格别名规则（strict aliasing）？

通过类型不兼容的左值访问对象是未定义行为（`char` 类型例外）。例如把 `float*` 强转为 `int*` 读取位模式是 UB，应使用 `memcpy` 或联合体（C 允许通过联合体做类型双关）。违反规则在 `-O2` 下可能产生错误代码；可用 `-fno-strict-aliasing` 临时规避。

Q32. 内存对齐是什么？为什么需要？

类型对齐要求通常等于其大小（或最大成员对齐）。未对齐访问在某些架构上会触发异常或变慢。可用 `_Alignof`、`_Alignas`、`aligned_alloc` 控制。SIMD 指令通常要求 16/32/64 字节对齐。

Q33. 柔性数组成员（flexible array member）是什么？

C99：结构体最后一个成员可声明为不完整数组 `char data[];`，分配时 `malloc(sizeof(struct S) + n)`。它不占 sizeof，用于变长报文/字符串。替代旧的 `data[1]` 或 `data[0]` 写法。

Q34. VLA 有什么问题？

变长数组分配在栈上，大小来自运行期值，容易栈溢出且无失败检测；C11 起为可选特性，许多代码规范（如 Linux 内核）禁止使用。应改用 `malloc` 或固定上限。

Q35. 复合字面量与指定初始化器是什么？

```
struct P { int x, y; };
struct P p = { .y = 2, .x = 1 };            // 指定初始化器
f(&(struct P){ .x = 3, .y = 4 });           // 复合字面量，块作用域内有效
int a[] = { [3] = 9 };
```

Q36. `char *s = "abc"` 和 `char s[] = "abc"` 有何不同？

前者 s 指向只读区的字符串字面量，修改是未定义行为（应写 `const char *`）；后者在栈/数据段上拷贝一份可修改的数组，`sizeof` 为 4。

Q37. 为什么不能返回局部变量的地址？

局部变量位于函数栈帧，函数返回后栈帧被回收，返回的地址悬空。可改为：返回堆内存、传入调用者缓冲区、返回 static 变量（非线程安全）或按值返回结构体。

Q38. qsort 的比较函数应如何写？

```
int cmp(const void *a, const void *b) {
    int x = *(const int*)a, y = *(const int*)b;
    return (x > y) - (x < y);        // 避免 x - y 溢出
}
qsort(arr, n, sizeof *arr, cmp);
```

qsort 不保证稳定。

Q39. offsetof 与 container_of 是什么？

`offsetof(type, member)`（`<stddef.h>`）给出成员偏移。`container_of(ptr, type, member)` 通过成员指针反推所属结构体地址：`(type*)((char*)ptr - offsetof(type, member))`，Linux 内核链表（`list_head`）依靠它实现侵入式容器。

Q40. 如何用 C 实现不透明类型/信息隐藏？

头文件只声明 `typedef struct Foo Foo;` 与函数原型，结构体定义放在 .c 中，调用者只持有指针。这样可以修改内部实现而不破坏 ABI。

1.3 动态内存管理

Q41. 栈和堆有什么区别？

栈：自动管理、LIFO、分配极快、大小有限（Linux 默认约 8MB）、生命周期随函数；堆：手动 `malloc/free`、大小只受系统限制、可能产生碎片、生命周期自行控制。

Q42. 进程地址空间布局是怎样的？

从低到高大致为：代码段（text，只读）→ 已初始化数据（data）→ 未初始化数据（bss）→ 堆（向上增长）→ 内存映射区（共享库、mmap）→ 栈（向下增长）→ 内核空间。

Q43. malloc、calloc、realloc、free 的区别？

`malloc(n)`：分配 n 字节，内容不确定；`calloc(c, n)`：分配 c*n 并清零，且检测乘法溢出；`realloc(p, n)`：调整大小，可能移动并拷贝；`free(p)`：释放。必须检查返回值是否为 NULL。

Q44. realloc 失败时怎么处理才安全？

失败返回 NULL 但原块仍有效。错误写法 `p = realloc(p, n);` 会丢失原指针造成泄漏。正确：

```
        void *tmp = realloc(p, n);
        if (!tmp) { /* 处理错误，p 仍有效 */ } else p = tmp;
```

Q45. malloc(0) 返回什么？free(NULL) 呢？

`malloc(0)` 实现定义：可能返回 NULL，也可能返回可 free 的唯一指针。`free(NULL)` 是安全的空操作。

Q46. double free 会怎样？

未定义行为，常表现为堆元数据被破坏、程序崩溃，甚至被利用（堆漏洞）。避免：释放后置 NULL、谁分配谁释放、明确所有权约定。

Q47. 怎样检测内存泄漏和越界？

Valgrind（Memcheck）、AddressSanitizer（`-fsanitize=address`）、LeakSanitizer、MSan（未初始化读）、UBSan（`-fsanitize=undefined`）、静态分析（clang-tidy、Coverity、cppcheck）、重载 malloc 的计数器。

Q48. 结构体对齐与填充怎么算？

每个成员按自身对齐要求放置，整体大小是最大成员对齐的倍数。`struct{char a; int b; char c;}` 一般为 12 字节，调整为 `{int b; char a; char c;}` 变为 8 字节。可用 `#pragma pack` 或 `__attribute__((packed))` 取消填充，但可能引发未对齐访问问题。

Q49. 什么是缓冲区溢出？如何防范？

向缓冲区写入超过其容量的数据，覆盖相邻内存（返回地址、变量）。防范：使用带长度的函数（`snprintf`、`fgets`）、检查边界、编译器防护（`-fstack-protector`、`_FORTIFY_SOURCE`、ASLR、NX、PIE）。

Q50. memcpy 与 memmove 的区别？

`memcpy` 要求源、目的不重叠，重叠是 UB；`memmove` 保证重叠情况下正确（内部按方向拷贝）。不确定是否重叠时使用 `memmove`。

Q51. 使用 memset 有哪些陷阱？

`memset` 按字节填充，只适合 0 或 -1（所有字节相同）。`memset(arr, 1, sizeof arr)` 对 int 数组得到 0x01010101 而不是 1。对含指针/浮点的结构体清零在多数平台可行，但严格说不保证表示为 NULL/0.0。编译器可能优化掉清除敏感数据的 `memset`，应使用 `memset_s` 或 `explicit_bzero`。

Q52. malloc 的底层实现原理？

glibc ptmalloc：小块从 brk/sbrk 扩展的堆上按 arena 管理，用 bins（fast/small/large/unsorted）维护空闲块；大块（默认≥128KB）直接 mmap。释放时合并相邻空闲块。其他分配器：jemalloc、tcmalloc、mimalloc 在多线程下性能更好。

Q53. 什么是内存碎片？如何缓解？

外部碎片：空闲内存总量足够但不连续；内部碎片：块内浪费。缓解：内存池、slab、arena（批量释放）、固定大小对象池、选用合适的分配器。

Q54. 什么是 arena/内存池分配器？

预先申请一大块内存，按线性或定长块分配，不单独释放，最后一次性释放整块。优点：分配极快、无碎片、释放简单。适合编译器 AST、请求级临时对象。

Q55. alloca 为什么不推荐？

在栈上动态分配，返回时自动释放，但不可移植、失败无法检测、容易栈溢出，也不能在需要作为参数表达式中安全使用。

Q56. use-after-free 是什么？

释放后继续使用该指针，读到被复用的数据或写坏别人的数据。是最常见的安全漏洞之一。防范：释放后置 NULL、所有权清晰、使用 ASan、引用计数。

Q57. 栈溢出是怎么发生的？

无限/过深递归、大局部数组、大 VLA。表现为 SIGSEGV。可用 `ulimit -s` 查看栈限制，改用迭代或堆内存。

Q58. calloc 与 malloc(c*n) 有何区别？

`calloc` 会检查 `c*n` 溢出并清零；手写乘法可能溢出导致分配过小，进而引发堆溢出。

1.4 字符串与标准库

Q59. strcpy、strncpy、snprintf 怎么选？

`strcpy` 不检查边界，不安全。`strncpy` 若源长度≥n 则不会补 `\0`，且在源较短时用 0 填满 n 字节（浪费），常被误用。BSD 的 `strlcpy`（glibc 2.38 起提供）或 `snprintf(dst, size, "%s", src)` 保证终止。C23 加入 `memccpy`，C11 附录 K 的 `_s` 系列很少被实现。

Q60. 手写 strlen、strcpy、memcpy。

```
          size_t my_strlen(const char *s){ const char *p=s; while(*p) p++; retur
n p-s; }
          char *my_strcpy(char *d, const char *s){ char *r=d; while((*d++=*s+
+)); return r; }
          void *my_memcpy(void *d, const void *s, size_t n){
              unsigned char *dp=d; const unsigned char *sp=s;
              while(n--) *dp++=*sp++; return d;
          }
```

面试要点：处理 NULL、返回值、重叠（memmove 从后向前）、对齐优化。

Q61. strcmp 的返回值和使用注意？

返回负数/0/正数，按无符号字符逐字节比较，不一定是 -1/1。不能用 `==` 比较字符串内容（比较的是地址）。

Q62. strtok 为什么不是线程安全的？

它在内部静态变量中保存位置，且会修改原字符串。线程安全替代：POSIX `strtok_r`，C11 附录 K 的 `strtok_s`。

Q63. sprintf 和 snprintf 的区别？

`sprintf` 不限制输出长度，易溢出；`snprintf(buf, n, ...)` 最多写 n-1 个字符并保证 `\0`，返回"如果空间足够本应写入的长度"，可据此判断是否截断。

Q64. atoi 与 strtol 的区别？

`atoi` 无法区分 0 与错误，溢出行为未定义。`strtol` 可指定进制、通过 `endptr` 检测非法字符、通过 `errno == ERANGE` 检测溢出，应优先使用。

Q65. 什么是格式化字符串漏洞？

把用户输入作为格式串：`printf(user)`。攻击者可用 `%x` 读栈、`%n` 写内存。应始终写 `printf("%s", user)`；编译器选项 `-Wformat-security`。

Q66. 为什么不能使用 gets？

`gets` 无法限制读入长度，必然导致缓冲区溢出，C11 已移除。使用 `fgets(buf, size, stdin)`，注意它会保留换行符。

Q67. C 里如何处理 UTF-8 / 宽字符？

`char` 数组可存放 UTF-8 字节序列，`strlen` 得到字节数而非字符数。`wchar_t` 宽度因平台不同（Windows 16 位、Linux 32 位）。C11 引入 `char16_t/char32_t` 与 `u8""`。处理 Unicode 通常借助 ICU、libunistring 等库。

Q68. 如何实现 itoa（整数转字符串）？

取模逐位生成再反转，注意负数和 `INT_MIN`（用无符号处理）：

```
void my_itoa(int v, char *s){
    unsigned u = v < 0 ? -(unsigned)v : v; char *p = s, *q;
    do { *p++ = '0' + u % 10; } while (u /= 10);
    if (v < 0) *p++ = '-';
    *p = 0;
    for (q = s, --p; q < p; q++, p--) { char t=*q; *q=*p; *p=t; }
}
```

1.5 预处理、编译与链接

Q69. 从源码到可执行文件经历哪些阶段？

预处理（宏展开、`#include`）→ 编译（生成汇编）→ 汇编（生成目标文件 .o）→ 链接（解析符号、重定位，生成可执行文件）。对应 `gcc -E / -S / -c`。

Q70. 头文件保护为什么必要？

防止同一头文件被重复包含导致重复定义。`#ifndef FOO_H / #define FOO_H / ... / #endif` 或非标准但广泛支持的 `#pragma once`。

Q71. `#include "x.h"` 与 `#include <x.h>` 的区别？

引号先在当前文件目录查找，再回退到系统路径；尖括号直接在系统/`-I` 路径查找。

Q72. 写宏时有哪些常见陷阱？

参数要加括号、整体要加括号、避免重复求值、多语句用 `do { ... } while (0)`：

```
#define SQUARE(x) ((x) * (x))
#define SWAP(a,b) do { typeof(a) t=(a); (a)=(b); (b)=t; } while (0)
```

`do/while(0)` 保证在 `if (x) MACRO(); else ...` 中语法正确。

Q73. # 与 ## 运算符是什么？

`#x` 把参数字符串化，`a ## b` 拼接标记。`#define STR(x) #x`；`#define VAR(n) var_##n`。若需先展开参数再字符串化，要套两层宏。

Q74. 如何编写可变参数函数？

```
#include <stdarg.h>
int sum(int n, ...) {
    va_list ap; va_start(ap, n); int s = 0;
    while (n--) s += va_arg(ap, int);
    va_end(ap); return s;
}
```

注意类型提升（`char`/`short` 按 int，`float` 按 double），且没有类型检查，格式类函数可加 `__attribute__((format(printf,1,2)))`。

Q75. 静态库与动态库的区别？

静态库（.a）在链接时拷贝进可执行文件，体积大但无运行时依赖；动态库（.so/.dll）运行时加载，多进程共享内存、可独立更新，但存在版本兼容与加载开销。静态库链接顺序敏感（依赖者在前）。

Q76. "undefined reference" 与 "multiple definition" 是什么错误？

前者：符号已声明但未找到定义（漏链接库/源文件、库顺序错）；后者：多个目标文件定义了同一全局符号（在头文件里定义变量/函数而未加 `static`/`inline`）。

Q77. 符号可见性与 weak 符号有什么用？

`-fvisibility=hidden` 配合 `__attribute__((visibility("default")))` 只导出必要符号，减小动态符号表。`__attribute__((weak))` 允许被强符号覆盖，常用于可选钩子和默认实现。

Q78. C99 的 inline 语义是怎样的？

`inline` 是对编译器的建议，C99 下 `inline` 定义需要在某个翻译单元提供 `extern inline` 外部定义，否则可能出现链接错误。通常写 `static inline` 放在头文件里。

Q79. 条件编译常用于什么？

平台差异（`#ifdef _WIN32`）、调试开关（`#ifdef DEBUG`）、功能裁剪、头文件保护。`#if defined(X) && X > 2` 可做版本判断。C23 有 `#elifdef`。

Q80. 常用的 GCC 属性有哪些？

`aligned`、`packed`、`noreturn`、`unused`、`format`、`constructor/destructor`、`cleanup`（实现类 RAII）、`always_inline`、`noinline`、`nonnull`。C23 标准化了 `[[nodiscard]]`、`[[maybe_unused]]`、`[[deprecated]]`、`[[noreturn]]`、`[[fallthrough]]`。

Q81. C 和 C++ 互相调用要注意什么？

C++ 会做名字改编，因此 C++ 里引用 C 函数要用 `extern "C"` 包裹；头文件常写成：

```
#ifdef __cplusplus
extern "C" {
#endif
/* 声明 */
#ifdef __cplusplus
}
#endif
```

Q82. 编译优化级别有什么区别？

`-O0` 不优化便于调试；`-O1/-O2` 常规优化（内联、循环优化、公共子表达式消除）；`-O3` 更激进（向量化、展开）；`-Os` 优化体积；`-Ofast` 放宽浮点语义。优化后调试信息可能与源码对不上，UB 代码行为也会被"暴露"。`-flto` 做链接时优化。

Q83. 如何生成调试信息并使用 gdb 调试崩溃？

`gcc -g -O0`；`gdb ./a.out core` 查看 `bt`；常用命令 `break`、`run`、`next/step`、`print`、`watch`、`x/16xb addr`、`info locals`。生产环境可用 `addr2line` 与符号文件解析堆栈。

1.6 结构体、联合与常见数据结构

Q84. struct 和 union 的区别？

struct 的成员各占独立空间；union 所有成员共用同一块内存，大小为最大成员，同一时刻只有一个成员有效。常用于变体类型（标签联合）、类型双关、节省内存。

Q85. 如何比较两个结构体？

不能直接用 `==`；`memcmp` 因填充字节内容不确定而不可靠，应逐成员比较（或先 `memset` 清零，但并不保证安全）。

Q86. 链表常见题：如何反转单链表？

```
          struct Node *reverse(struct Node *head){
              struct Node *prev = NULL;
              while (head) { struct Node *next = head->next; head->next = prev;
prev = head; head = next; }
              return prev;
          }
```

时间 O(n)，空间 O(1)。

Q87. 如何判断链表是否有环并找到入口？

快慢指针：fast 每次两步、slow 一步，相遇则有环；然后一个指针回到头，两者同速前进，再次相遇处即入口。

Q88. 如何用 C 实现"面向对象"？

结构体 + 函数指针表（vtable）：对象首成员放指向 ops 表的指针，通过 `obj->ops->method(obj)` 调用；继承用"首成员嵌入父结构"，再配合 `container_of` 转换。Linux 内核、GObject 大量采用。

Q89. setjmp/longjmp 是做什么的？

`setjmp` 保存当前执行上下文，`longjmp` 非局部跳转回去，可模拟异常/错误退出。注意：跳过了局部对象的清理，局部非 volatile 变量的值在 longjmp 后不确定。

Q90. errno 如何使用？

库函数/系统调用失败时设置的全局（线程局部）错误码，成功时不会被清零，因此应先检查返回值，再读取 errno，并立即保存；用 `strerror` / `perror` 输出文本。

Q91. fopen/fread 与 open/read 有何区别？

前者是 C 标准库，带用户态缓冲，可移植；后者是 POSIX 系统调用，无缓冲、返回文件描述符、支持更细的控制（`O_DIRECT`、非阻塞等）。二者可通过 `fdopen` / `fileno` 互转。

Q92. 标准 I/O 的三种缓冲模式？

无缓冲（stderr）、行缓冲（连接终端的 stdout）、全缓冲（文件、管道）。`fflush` 刷新，`setvbuf` 设置。程序崩溃时未刷新的 stdout 数据会丢失，是常见"日志没打出来"的原因。

Q93. 文本模式和二进制模式有何区别？

在 Windows 上文本模式会转换换行（`\r\n` ↔ `\n`）并把 Ctrl-Z 视为 EOF；POSIX 系统无区别。读写结构体/图片应使用 `"rb"` / `"wb"`。

Q94. 如何实现一个简单的哈希表？

数组 + 链地址法：`hash(key) % cap` 定位桶；负载因子超过阈值（如 0.75）后扩容并 rehash；注意字符串哈希（如 djb2/FNV）、冲突处理、键拷贝、释放。开放寻址法缓存友好但需处理删除标记。

1.7 并发与系统编程

Q95. fork 与 exec 的区别？

`fork` 复制当前进程（写时复制），返回 0 给子进程、子进程 PID 给父进程；`exec` 系列用新程序替换当前进程映像（PID 不变）。典型 shell：fork → 子进程 exec → 父进程 wait。

Q96. 什么是僵尸进程和孤儿进程？

僵尸：子进程已退出但父进程未 `wait`，仍占用进程表项。孤儿：父进程先退出，被 init(PID 1) 收养。避免僵尸：`waitpid`、处理 `SIGCHLD`、双 fork 或 `SA_NOCLDWAIT`。

Q97. 信号处理函数中应该注意什么？

只能调用异步信号安全函数（`write`、`_exit`、`sigaction` 等），不能调用 `malloc`、`printf`。访问共享变量应使用 `volatile sig_atomic_t`。推荐使用 `sigaction` 而非 `signal`，必要时使用 `signalfd` 或自管道转换为事件循环处理。

Q98. pthread 基本使用与常见同步原语？

`pthread_create/join/detach`，互斥量 `pthread_mutex_t`，条件变量 `pthread_cond_t`，读写锁，自旋锁，信号量，屏障。互斥量必须配对加解锁，条件变量必须与互斥量一起使用。

Q99. 条件变量为什么要用 while 循环检测条件？

存在"虚假唤醒"（spurious wakeup），且被唤醒时条件可能已被其他线程改变，所以必须用 `while (!cond) pthread_cond_wait(&c, &m);`。

Q100. 死锁的四个必要条件及预防？

互斥、占有并等待、不可抢占、循环等待。预防：统一加锁顺序、一次获取所有锁、`trylock` 加超时回退、减少锁粒度、避免在持锁时调用外部回调。

Q101. 什么是线程安全与可重入？

线程安全：多线程同时调用结果正确（可通过锁实现）；可重入：不依赖全局/静态状态，可在被中断后再次进入（更强）。可重入函数一定线程安全，反之不一定。例如 `strtok` 不可重入，`strtok_r` 可重入。

Q102. C11 原子操作与内存序有哪些？

`<stdatomic.h>` 提供 `atomic_int`、`atomic_fetch_add`、`atomic_compare_exchange_strong` 等。内存序：`relaxed`（仅原子性）、`acquire`、`release`、`acq_rel`、`seq_cst`（默认，全局顺序）。典型：生产者 release 写标志，消费者 acquire 读标志，从而保证之前的写入可见。

Q103. 什么是 false sharing？

不同线程频繁写位于同一缓存行（常见 64 字节）的不同变量，导致缓存行在核间反复失效。解决：用 `_Alignas(64)` 或填充使热点变量独占缓存行。

Q104. 进程间通信有哪些方式？

管道/命名管道、消息队列、共享内存（最快，需要同步）、信号量、信号、Unix 域套接字、TCP/UDP 套接字、`mmap` 文件映射。

Q105. select、poll、epoll 的区别？

select：fd 数量受 `FD_SETSIZE`（1024）限制，每次调用都要拷贝并遍历集合；poll：无数量上限但仍需线性遍历；epoll（Linux）：内核维护就绪列表，`epoll_wait` 只返回就绪事件，复杂度与活跃连接数相关，支持水平触发（LT）与边缘触发（ET，需要非阻塞 IO 并读到 EAGAIN）。

Q106. 阻塞与非阻塞、同步与异步 IO 的区别？

阻塞/非阻塞描述调用在数据未就绪时是否挂起；同步/异步描述结果通知方式：同步 IO 需要调用者自己读取数据（含 epoll 的 IO 多路复用），异步 IO（如 Linux `io_uring`、Windows IOCP、POSIX AIO）由内核完成拷贝后再通知。

Q107. mmap 有什么用途？

把文件或匿名内存映射到进程地址空间：读写大文件、进程间共享内存、按需分页加载、实现 malloc 大块分配。注意缺页、同步（`msync`）、文件被截断时访问会触发 SIGBUS。

Q108. 什么是写时复制（Copy-On-Write）？

`fork` 后父子进程共享物理页，标记为只读，任意一方写入时才复制该页。减少 fork 的开销，也是 Redis RDB 快照等机制的基础。

Q109. 如何防止头文件中的静态变量被重复定义或造成多份副本？

在头文件中只放 `extern` 声明，在一个 .c 文件中定义；如果在头文件里写了 `static` 变量，每个包含它的翻译单元都会得到一份独立的副本，通常不是预期行为。

Q110. C 语言中常见的未定义行为有哪些？

有符号溢出、空指针解引用、越界访问、使用未初始化变量（部分情形）、违反严格别名、修改字符串字面量、同一对象在序列点间多次修改、除零、移位位数≥类型宽度或为负、重叠使用 `memcpy`、数据竞争。建议开启 `-Wall -Wextra`，并在测试中使用 UBSan/ASan。

第二部分：C++

2.1 语言基础

Q1. C 和 C++ 的主要区别？

C++ 支持面向对象、泛型（模板）、异常、重载、引用、命名空间、RAII、标准库容器与算法。C++ 更强调类型安全（`void*` 不能隐式转换、`const` 语义更严格）。两者的 ABI 与名字改编不同，混用需 `extern "C"`。C++ 不是 C 的严格超集（如 VLA、复合字面量、指定初始化器在 C++20 前不支持）。

Q2. 引用和指针有什么区别？

引用是对象的别名：必须初始化、不能重新绑定、没有空引用、没有引用的引用（折叠规则除外）、不能取"引用的地址"（取到的是被引用对象地址）。指针是独立对象，可为空、可重新指向、可做算术。接口设计中：必须有值用引用，可选值用指针或 `std::optional`。

Q3. const 有哪些用法？const 成员函数与 mutable？

修饰变量、指针（顶层/底层 const）、引用、成员函数。`const` 成员函数承诺不修改对象的逻辑状态，this 变为 `const T*`，只能调用其他 const 成员函数。`mutable` 成员即使在 const 函数中也可修改，常用于缓存、互斥量。

Q4. const、constexpr、consteval、constinit 的区别？

`const`：运行期不可变，可在运行期初始化。`constexpr`：可在编译期求值（变量必须是常量表达式；函数在常量上下文中编译期求值，否则也可运行期调用）。`consteval`（C++20）：立即函数，必须在编译期求值。`constinit`（C++20）：保证静态/线程存储期变量在编译期初始化，但变量本身可变，用于避免静态初始化顺序问题。

Q5. 类的 static 成员有哪些规则？

静态数据成员属于类而非对象，需在类外定义（C++17 起可用 `inline static` 在类内定义）；静态成员函数没有 this 指针，不能访问非静态成员。

Q6. inline 在 C++ 中是什么含义？

现代含义主要是允许在多个翻译单元中重复定义而不违反 ODR（单一定义规则），内联展开只是建议。类内定义的成员函数隐式 inline；C++17 的 `inline` 变量允许在头文件中定义全局变量。

Q7. auto、decltype、decltype(auto) 的区别？

`auto` 按模板参数推导规则推导（忽略引用和顶层 const）；`decltype(expr)` 保留表达式的精确类型（含引用、const），`decltype((x))` 对变量加括号得到引用；`decltype(auto)` 用 decltype 规则推导初始化表达式，常用于完美转发返回类型。

Q8. nullptr 比 NULL 好在哪里？

`NULL` 通常是整数 0，会在重载决议中选择 `int` 版本；`nullptr` 类型为 `std::nullptr_t`，只能转换为指针类型，避免二义性。

Q9. enum class 与传统枚举的区别？

`enum class` 作用域化（需 `Color::Red`）、不会隐式转换为整数、可指定底层类型（`enum class E : uint8_t`）、可前向声明。传统枚举污染外层命名空间并可隐式转 int。C++23 提供 `std::to_underlying`。

Q10. 四种类型转换及适用场景？

`static_cast`：编译期检查的常规转换、向上/向下（不检查）转换；`dynamic_cast`：多态类型向下转换，运行期检查，失败返回 nullptr（指针）或抛 `bad_cast`（引用），依赖 RTTI；`const_cast`：移除/添加 const（修改真正 const 对象是 UB）；`reinterpret_cast`：位级重新解释，高度不可移植。尽量避免 C 风格转换。

Q11. 统一初始化（花括号初始化）有什么好处与坑？

`T x{...}` 统一初始化语法，禁止窄化转换（`int x{3.5}` 报错）。坑：`std::vector<int> v{3, 4}` 优先匹配 `initializer_list`，得到 2 个元素；`vector<int> v(3, 4)` 才是 3 个 4。

Q12. 值类别有哪些？lvalue、rvalue、xvalue、prvalue 是什么？

C++11 起分为 lvalue（有身份、不可移动）、xvalue（有身份、可移动，如 `std::move(x)`）、prvalue（纯右值，无身份，如字面量、临时对象）。glvalue = lvalue ∪ xvalue，rvalue = xvalue ∪ prvalue。C++17 保证拷贝省略，prvalue 不再物化为临时对象直到需要。

Q13. 什么是 ODR（单一定义规则）？

同一个实体在整个程序中只能有一个定义（内联函数/类/模板可在不同翻译单元重复定义，但定义必须相同）。违反 ODR 是"无需诊断"的未定义行为，常见于头文件里定义非 inline 全局函数/变量。

Q14. 命名空间与 ADL（参数依赖查找）是什么？

调用无限定函数名时，除普通查找外，还会在实参类型所属的命名空间中查找，因此 `std::cout << x` 能找到 `std::operator<<`。ADL 也是自定义 `swap` 的惯用法：`using std::swap; swap(a, b);`。避免在头文件中写 `using namespace std;`。

Q15. 范围 for 与结构化绑定如何使用？

```
for (const auto& [k, v] : m) { ... }   // C++17 结构化绑定
for (auto& x : vec) x *= 2;
```

范围 for 展开为 begin()/end() 迭代，不要在遍历中做会使迭代器失效的容器修改（如 vector 的 push_back）。

Q16. 函数重载解析如何进行？

按"精确匹配 > 提升 > 转换 > 用户转换"选择最佳可行函数，不能仅以返回值区分重载。默认参数、模板和非模板同时存在时，非模板优先。名字改编使得重载在链接层可区分。

Q17. 默认参数与重载如何取舍？

默认参数在编译期替换，写在声明处，对虚函数其值由静态类型决定（不会随动态类型变化，容易出错）。需要不同行为时用重载。

Q18. 运算符重载有哪些原则？

保持语义直观；赋值、下标、函数调用、`->` 必须是成员；对称运算（`+`、`==`）宜为非成员/友元；`operator<<` 为非成员；重载 `&&`、`||`、`,` 会失去短路/顺序语义，应避免；C++20 可用 `operator<=>` 自动生成比较运算符。

Q19. 用户定义字面量是什么？

`constexpr Meters operator""_m(long double v);` 允许 `5.0_m`。标准库里有 `"abc"s`、`10ms`、`1i`。

Q20. std::string、std::string_view 的区别与风险？

`string` 拥有数据；`string_view`（C++17）是对一段连续字符的非拥有视图，拷贝廉价。风险：悬空——不要把 string_view 绑定到临时 string 并保存，也不要假定以 `\0` 结尾。

2.2 类与对象

Q21. 构造和析构的调用顺序？

构造：基类（按声明顺序，虚基类最先）→ 成员（按声明顺序）→ 派生类构造体。析构顺序与构造完全相反。

Q22. 成员初始化列表有什么作用？

初始化 const 成员、引用成员、没有默认构造的成员/基类；避免"先默认构造再赋值"的开销。初始化顺序由成员声明顺序决定，而非列表书写顺序。

Q23. explicit 关键字的作用？

禁止单参数（或带默认值）构造函数的隐式转换，也可用于转换运算符（`explicit operator bool`）。C++20 的 `explicit(bool)` 可按条件启用。

Q24. 什么是 Rule of 0/3/5？

Rule of 3：若自定义了析构、拷贝构造、拷贝赋值之一，通常三个都需要。Rule of 5：再加移动构造与移动赋值。Rule of 0：优先使用 RAII 成员（`vector`、`unique_ptr`），让编译器生成所有特殊成员函数。

Q25. 拷贝构造和拷贝赋值的区别？如何写安全的赋值运算符？

拷贝构造创建新对象，赋值是覆盖已存在对象。赋值需处理自赋值与异常安全，推荐 copy-and-swap：

```
T& operator=(T other) noexcept { swap(*this, other); return *this; }
```

Q26. 深拷贝与浅拷贝？

浅拷贝按成员复制，若成员含指针则多个对象共享同一资源，导致重复释放。深拷贝复制所指资源。使用 `vector`、`string`、智能指针可避免手写。

Q27. `= default` 与 `= delete` 有什么用？

`= default` 显式要求编译器生成特殊成员函数；`= delete` 禁用函数（不可拷贝类型、禁止隐式转换的重载、禁止某些模板实例化）。

Q28. 委托构造和继承构造是什么？

委托构造：`A(int x) : A(x, 0) {}`；继承构造：`using Base::Base;` 把基类构造函数引入派生类。

Q29. 虚函数是如何实现的？

大多数实现为每个含虚函数的类生成虚函数表（vtable），对象内含指向 vtable 的指针（vptr）。调用虚函数时通过 vptr 间接查表，因此有一次额外访存，并且通常阻碍内联（除非能去虚化）。多重继承时对象内可有多个 vptr。

Q30. 为什么基类析构函数应声明为 virtual？

通过基类指针 delete 派生类对象时，若析构非虚，则只调用基类析构，派生部分泄漏（未定义行为）。多态基类应有虚析构或保护的非虚析构。

Q31. 纯虚函数与抽象类是什么？

`virtual void f() = 0;` 使类成为抽象类，不能实例化，派生类必须实现才能实例化。纯虚函数也可以提供定义（析构函数常用）。

Q32. override 与 final 的作用？

`override` 让编译器检查确实重写了基类虚函数（拼写或签名错误会报错）；`final` 禁止进一步重写或继承，可帮助去虚化优化。

Q33. 构造/析构函数里调用虚函数会怎样？

此时对象的动态类型是当前正在构造/析构的类，所以不会调用派生类的重写版本；如果调用纯虚函数且无定义则出错。

Q34. 多重继承和菱形继承问题如何解决？

菱形继承会产生基类的两份子对象，访问二义；使用虚继承（`class B : virtual public A`）使只保留一份，代价是额外间接与由最派生类初始化虚基类。更推荐组合 + 接口类。

Q35. 什么是对象切片（slicing）？

把派生类对象按值赋给基类对象时，派生部分被丢弃，且丢失多态。应使用指针或引用传递多态对象。

Q36. friend 的作用与代价？

让某函数/类访问私有成员，破坏封装但常用于运算符重载与紧密协作的类。友元关系不传递也不继承。

Q37. 编译期多态与运行期多态？CRTP 是什么？

运行期多态：虚函数。编译期多态：模板、重载。CRTP（奇异递归模板模式）：`class D : public Base<D>`，基类通过 `static_cast<D*>(this)` 调用派

生类实现，无虚调用开销，常用于混入功能和静态接口。C++23 的 deducing this 简化了此类写法。

Q38. 虚函数调用一定不能内联吗？

通过指针调用的虚函数通常经由 vtable 间接调用而无法内联；若编译器能确定动态类型（局部对象、`final`、PGO 去虚化、LTO）则可内联。

Q39. 空类的大小是多少？什么是 EBO？

空类 `sizeof` 至少为 1，以保证不同对象有不同地址。作为基类时，空基类优化（EBO）可让其不占空间；C++20 的 `[[no_unique_address]]` 对成员实现类似效果。

Q40. 如何计算类对象大小？

非静态数据成员 + 对齐填充 + vptr（有虚函数时）+ 虚基类指针。静态成员和成员函数不占对象空间。

Q41. 重载（overload）、重写（override）、隐藏（hide）的区别？

重载：同作用域同名不同参；重写：派生类对基类虚函数的同签名覆盖；隐藏：派生类的同名函数（无论签名）隐藏基类同名函数，需 `using Base::f;` 引入。

Q42. RVO / NRVO 与保证拷贝省略（C++17）是什么？

返回值优化：直接在调用者的目标位置构造返回对象，省去拷贝/移动。C++17 起返回 prvalue（如 `return T{}`）强制省略；具名对象的 NRVO 仍是可选。不要对返回的局部变量写 `return std::move(x)`，会抑制 NRVO。

Q43. 类型转换运算符与转换构造函数？

`operator T() const` 允许对象隐式转换为 T；单参构造函数允许隐式转换到类类型。隐式转换易引发歧义，建议加 `explicit`。

Q44. this 指针和成员函数指针是什么？

this 是指向当前对象的隐含指针参数。成员函数指针 `R (C::*pm)(Args...)` 需要通过对象调用：`(obj.*pm)(args)`，其大小可能大于普通指针（含虚函

数偏移信息）。`std::invoke` 可统一调用方式。

Q45. Pimpl 惯用法是什么？

把实现细节放在不透明的私有实现类中，头文件只含 `std::unique_ptr<Impl>`。优点：减少编译依赖、稳定 ABI；代价：额外间接与堆分配。注意 `unique_ptr<Impl>` 需在 Impl 完整的翻译单元中定义析构。

2.3 内存管理与智能指针

Q46. new/delete 与 malloc/free 的区别？

`new` 先分配再调用构造函数，失败抛 `bad_alloc`；`malloc` 只分配原始内存，失败返回 NULL。`delete` 先析构再释放。二者不可混用，数组要用 `new[]` / `delete[]` 配对。

Q47. 为什么 new[] 必须配 delete[]？

`new[]` 可能额外存储元素个数，`delete[]` 依此逐个调用析构；错配是未定义行为。

Q48. placement new 是什么？

在已有内存上构造对象：`new (buf) T(args);`，不分配内存。使用后需手动调用析构 `p->~T()`。用于内存池、`std::vector` 的实现、`std::optional`。

Q49. 什么是 RAII？

资源获取即初始化：把资源（内存、文件、锁、套接字）的生命周期绑定到对象，构造时获取、析构时释放，从而在异常和提前返回时自动清理。例：`lock_guard`、`unique_ptr`、`fstream`。

Q50. unique_ptr 的特点？

独占所有权、不可拷贝、可移动，零开销（默认删除器时大小等于裸指针）。用 `std::make_unique`（C++14）创建。可作为工厂函数返回值，也可管理数组 `unique_ptr<T[]>`。

Q51. shared_ptr 如何实现？线程安全吗？

由指向对象的指针和指向控制块的指针组成；控制块含强引用计数、弱引用计数、删除器、分配器。引用计数的增减是原子的，因此多个线程各自持有副本并拷贝/销毁是安全的；但同一个 shared_ptr 对象被多线程同时读写不是安全的（需 `atomic<shared_ptr>`，C++20），所指对象本身的访问也需要同步。

Q52. weak_ptr 是做什么的？

不增加强引用计数的观察者，用 `lock()` 获得 shared_ptr（对象已销毁则为空）。解决 shared_ptr 循环引用（如父子互相持有），也用于缓存和观察者模式。

Q53. make_shared 比 new 好在哪里？

一次分配同时放对象和控制块，更快更省，异常安全（避免 `f(shared_ptr<T>(new T), g())` 的泄漏）。缺点：对象内存要等所有 weak_ptr 也释放后才回收；不能使用自定义删除器。

Q54. enable_shared_from_this 有什么用？

让对象在内部安全获得指向自己的 shared_ptr（`shared_from_this()`），避免对同一裸指针创建多个独立控制块导致重复释放。对象必须已被 shared_ptr 管理。

Q55. 如何给智能指针指定自定义删除器？

`unique_ptr<FILE, decltype(&fclose)> fp(fopen(...), &fclose);`，删除器类型是 unique_ptr 类型的一部分；`shared_ptr` 的删除器存于控制块，不影响类型。

Q56. 如何排查 C++ 内存泄漏？

AddressSanitizer / LeakSanitizer、Valgrind、重载全局 `operator new` 统计、堆分析器（heaptrack、jemalloc profiler）。在代码层面优先使用 RAII，避免裸 new/delete。

Q57. 自定义分配器与 std::pmr 是什么？

容器的第二模板参数可替换分配器。C++17 `std::pmr::polymorphic_allocator` 通过运行期 `memory_resource`（如 monotonic_buffer_resource）让容器类型不变而切换分配策略，适合 arena 式分配。

Q58. alignas 与对齐分配？

`alignas(64) struct S{...};` 指定对齐；C++17 起 `new` 支持过度对齐类型；`std::aligned_alloc` 分配对齐内存。用于避免 false sharing 与 SIMD。

Q59. 什么是悬空引用与迭代器失效？

引用/指针/迭代器指向的对象已销毁或容器重新分配后使旧迭代器失效。常见：`vector::push_back` 扩容、返回局部变量引用、持有临时对象的 `string_view`、range-for 绑定临时对象的子对象（C++23 修复了部分情形）。

Q60. 栈对象与堆对象如何选择？

默认用自动存储期（栈）对象；需要多态、生命周期超出作用域、大对象或运行期确定大小时使用堆（通过智能指针/容器）。

2.4 移动语义与模板

Q61. 右值引用和移动语义解决了什么问题？

避免不必要的深拷贝：临时对象的资源可被"窃取"。移动构造把资源指针转移并将源置为有效但未指定的状态（通常为空）。

Q62. std::move 做了什么？

只是无条件转换为右值引用（`static_cast<T&&>`），本身不移动任何东西。真正的移动发生在被匹配到的移动构造/赋值函数里。对 const 对象 move 会退化为拷贝。

Q63. 万能引用与 std::forward（完美转发）？

`template<class T> void f(T&& x)` 中的 `T&&` 是转发引用：传入左值时 T 推导为 `U&`，传入右值时为 `U`。`std::forward<T>(x)` 保持原有值类别地转发给下游函数。`auto&&` 也是转发引用。

Q64. 引用折叠规则是什么？

`T& &`、`T& &&`、`T&& &` 折叠为 `T&`；仅 `T&& &&` 折叠为 `T&&`。这是转发引用与 `std::move` / `forward` 工作的基础。

Q65. 移动构造函数为什么应标 noexcept？

`std::vector` 扩容时若移动构造可能抛异常，则为保证强异常安全而回退使用拷贝（`move_if_noexcept`）。因此没有 `noexcept` 的移动会让容器性能退化。

Q66. 函数模板与类模板如何特化？

函数模板只支持全特化（偏特化用重载或 `if constexpr` 替代）；类模板支持全特化与偏特化；变量模板（C++14）亦可特化。

Q67. 什么是 SFINAE？

替换失败不是错误：模板参数替换产生无效类型时，该重载从候选集中移除而不是报错。常配合 `std::enable_if_t` 或 `decltype` 表达式 SFINAE 约束模板。C++20 的 concepts 更清晰。

Q68. C++20 Concepts 是什么？

对模板参数的具名约束：

```
          template<std::integral T> T add(T a, T b) { return a + b; }
          template<class T> requires std::totally_ordered<T> bool less(T a, T
b);
```

错误信息更友好，重载决议可按约束的"更特化"选择。

Q69. 可变参数模板与折叠表达式？

```
template<class... Ts> auto sum(Ts... xs) { return (xs + ...); }  // C++17 折叠表达式
```

`sizeof...(Ts)` 取参数个数，递归展开或折叠展开参数包。

Q70. 模板为什么通常要放在头文件中？

编译器实例化时需要看到完整定义。若想分离，可显式实例化（`template class Foo<int>;`）或使用 C++20 模块。滥用模板会导致代码膨胀与编译变慢，可通过 extern template 减少重复实例化。

Q71. type_traits 有什么用？

编译期查询或变换类型：`is_integral`、`is_same`、`remove_reference`、`decay`、`conditional`、`is_trivially_copyable`。是 SFINAE、concepts 和元编程的基础，C++17 的 `_v`、`_t` 后缀简化写法。

Q72. if constexpr 的作用？

编译期条件分支，未选中的分支不会被实例化，可替代大量 SFINAE：

```
template<class T> void f(T v){ if constexpr (std::is_pointer_v<T>) use(*v); else use(v); }
```

Q73. 类模板参数推导（CTAD）和模板模板参数？

C++17 起 `std::pair p{1, 2.0};`、`std::vector v{1,2,3};` 可省略模板实参。模板模板参数允许把模板作为参数：`template<template<class> class C> struct X{};`。

Q74. lambda 表达式的捕获方式与陷阱？

`[=]` 按值、`[&]` 按引用、`[x, &y]` 混合、`[this]`、`[*this]`（C++17 拷贝对象）、初始化捕获 `[p = std::move(ptr)]`（C++14）。按引用捕获的局部变量在 lambda 逃逸后悬空；`[=]` 在成员函数里实际捕获的是 this。泛型 lambda 用 `auto` 参数；`mutable` 允许修改按值捕获的副本。

Q75. std::function、函数指针、lambda 如何选择？

函数指针：无状态、最轻；lambda：每个有独立类型，可内联，零开销；`std::function`：类型擦除，可存任意可调用对象，但可能堆分配并有间接调用开销。接口里用模板参数或 `function_ref`（C++26）/`std::move_only_function`（C++23）。

Q76. 为什么需要 typename 和 template 消歧义关键字？

在依赖名称中，编译器无法判断 `T::value_type` 是类型还是值，需写 `typename T::value_type`；调用依赖模板成员需要 `obj.template f<int>()`。C++20 在更多上下文中放宽了 typename 要求。

2.5 STL 容器与算法

Q77. vector 的扩容机制是什么？

容量不足时重新分配更大的内存（GCC 为 2 倍，MSVC 约 1.5 倍），移动/拷贝元素后释放旧内存，因此 push_back 摊还 O(1)。`reserve` 预分配避免多次扩容；`shrink_to_fit` 请求释放多余容量。

Q78. `vector<bool>` 有什么问题？

它是按位压缩的特化，`operator[]` 返回代理对象而非 `bool&`，不满足容器要求，不能取元素地址。可用 `vector<char>`、`deque<bool>` 或 `bitset` 代替。

Q79. vector、deque、list、forward_list 的区别与选择？

vector：连续存储，随机访问和缓存友好，是默认首选；deque：分段连续，两端 O(1) 插入，迭代器稳定性稍好；list：双向链表，任意位置已知迭代器处 O(1) 插入，但缓存差、额外内存大；forward_list：单链表。大多数情况下 vector 胜出。

Q80. map 和 unordered_map 的区别？

map（红黑树）：有序、O(log n)、迭代器稳定；unordered_map（哈希表，桶+链）：平均 O(1)，最坏 O(n)，无序，rehash 会使迭代器失效（引

用和指针仍有效）。键需要可比较或可哈希。

Q81. map::operator[] 有什么副作用？

键不存在时会插入默认值元素，所以在 const map 上不可用，且不应用来判断键是否存在，应使用 `find`、`contains`（C++20）或 `at`（越界抛异常）。

Q82. unordered_map 的实现与自定义哈希？

标准库实现通常是桶数组加链表，负载因子默认 1.0，超出时 rehash。自定义键需提供 `std::hash` 特化或哈希函数对象及 `operator==`：

```
struct H { size_t operator()(const P& p) const { return std::hash<int>()(p.x) ^ (std::hash<int>()(p.y) << 1); } };
```

注意混合哈希值质量，防止恶意输入造成冲突退化。

Q83. priority_queue 的底层实现？

默认是基于 vector 的二叉堆（`push_heap/pop_heap`），默认最大堆，传 `greater<>` 得到最小堆。不支持遍历、随机删除和修改优先级；`top` O(1)，`push/pop` O(log n)。

Q84. array、span、string_view 的用途？

`std::array`：固定大小栈数组，带容器接口；`std::span`（C++20）：对连续序列的非拥有视图（指针+长度）；`string_view`：字符串视图。都应注意被引用数据的生命周期。

Q85. 迭代器的分类？

输入、输出、前向、双向、随机访问、连续（C++20）。算法按所需迭代器类别约束，如 `std::sort` 需要随机访问迭代器，故不能用于 `list`（用 `list::sort`）。

Q86. sort、stable_sort、nth_element、partial_sort 的区别？

`sort`：平均 O(n log n)，不稳定，通常实现为 introsort；`stable_sort`：稳定，O(n log n)，可能用额外内存；`nth_element`：O(n) 找到第 k 小并划分；`partial_sort`：只排前 k 个。比较函数必须满足严格弱序，否则可能崩溃。

Q87. erase-remove 惯用法是什么？

`std::remove/remove_if` 只是把保留的元素前移并返回新末尾，不改变容器大小，需配合 `erase`：`v.erase(std::remove(v.begin(), v.end(), x), v.end());`。C++20 提供 `std::erase(v, x)` 和 `std::erase_if`。

Q88. emplace_back 与 push_back 的区别？

`emplace_back(args...)` 原地构造元素，避免一次临时对象的构造/移动；`push_back` 接收已构造对象。差别在简单类型上可忽略；`emplace` 可能触发显式构造函数，不当使用会降低可读性。

Q89. 各容器的迭代器失效规则？

vector：扩容使全部失效，插入/删除使该点之后失效；deque：中间插入使全部失效，两端插入使迭代器失效但引用有效；list/map/set：仅被删除元素失效；unordered 容器：rehash 使迭代器失效。遍历中删除应使用 `it = c.erase(it)`。

Q90. optional、variant、any 的用途？

`optional<T>`：可能为空的值；`variant<Ts...>`：类型安全的联合体，配合 `std::visit`；`any`：类型擦除的任意值（运行期 `any_cast`）。优先使用 optional 和 variant，因为有静态类型信息。

Q91. C++20 Ranges 带来了什么？

惰性视图与管道语法：`v | std::views::filter(f) | std::views::transform(g)`，以及接受整个范围的算法 `std::ranges::sort(v)`、投影（projection）。视图不拥有数据，需注意生命周期。

Q92. tuple、pair 与结构化绑定的使用？

`std::tie(a, b) = f();` 解包；C++17 `auto [a, b] = f();`。`std::make_tuple`、`std::get<I>`、`std::apply` 可操作元组。字段多时优先使用有名字的结构体。

Q93. 三路比较运算符 <=> 是什么？

C++20：`auto operator<=>(const T&) const = default;` 自动生成 `<`、`<=`、`>`、`>=`（与默认 `==`）。返回 `strong_ordering` / `weak_ordering` / `partial_ordering`。

Q94. lower_bound / upper_bound / binary_search 的区别？

`lower_bound`：第一个 ≥ x 的位置；`upper_bound`：第一个 > x 的位置；`binary_search`：仅返回是否存在；`equal_range` 返回两者。要求范围已按相同比较器排序。

2.6 并发与内存模型

Q95. std::thread 的 join 与 detach？

线程对象析构前必须 join 或 detach，否则 `std::terminate`。`detach` 后线程独立运行，须保证其访问的数据一直有效。C++20 的 `std::jthread` 在析构时自动 join 并支持 `stop_token`。

Q96. mutex 相关封装怎么选？

`lock_guard`：最简单 RAII；`unique_lock`：可延迟加锁、解锁、配合条件变量；`scoped_lock`（C++17）：一次锁多个互斥量并避免死锁；`shared_mutex` + `shared_lock`：读写锁。

Q97. condition_variable 的使用与虚假唤醒？

等待者持有 `unique_lock`，使用带谓词的 `cv.wait(lock, pred)`（内部循环检查），通知者修改状态后 `notify_one/all`。必须在共享状态上加锁修改，否则会丢失唤醒。

Q98. atomic 与 memory_order？

`std::atomic<T>` 提供原子读改写与 `compare_exchange`。内存序：`relaxed`（只保证原子性）、`acquire/release`（建立同步关系）、`acq_rel`、`seq_cst`（默认，全局一致顺序）。除非有性能证据，默认使用 `seq_cst`。

Q99. 数据竞争是什么？

两个线程并发访问同一内存位置，至少一个是写，且没有同步关系——未定义行为。检测：ThreadSanitizer（`-fsanitize=thread`）。`volatile` 不能解决数据竞争。

Q100. future、promise、async、packaged_task 的区别？

`promise` / `future` 是一次性共享状态的两端；`async` 启动异步任务并返回 future（策略 `launch::async` 或 deferred）；`packaged_task` 包装可调用对象并产出 future。`future` 析构（来自 `async` 的）可能阻塞等待。工业场景常用线程池 + 自己的任务队列。

Q101. 如何避免死锁？

统一加锁顺序、使用 `std::scoped_lock` / `std::lock` 一次性获取、减少持锁时间、避免持锁调用未知代码、使用 `try_lock` 回退。

Q102. thread_local 的用途？

每个线程拥有一份独立变量，用于线程私有缓存、随机数引擎、错误码等。生命周期随线程。

Q103. C++20 协程是什么？

函数体含 `co_await` / `co_yield` / `co_return` 即协程，编译器把它变换成状态机，帧通常堆分配。语言只提供机制，需要 promise_type、awaitable 等；库层面如 cppcoro、Boost.Asio 的 awaitable，C++23 有 `std::generator`。

Q104. C++ 内存模型中的 happens-before 是什么？

由 sequenced-before（同线程顺序）与 synchronizes-with（release 写被 acquire 读观察到）、线程创建/join、互斥量加解锁组合而成的偏序关系。若写操作 happens-before 读操作，则读能看到该写入；否则存在数据竞争。

Q105. 单例模式的线程安全写法？

C++11 起函数内静态局部变量初始化是线程安全的（Meyers Singleton）：

```
Foo& instance() { static Foo f; return f; }
```

双重检查锁需配合原子或 `std::call_once`。

Q106. C++20 的 latch、barrier、semaphore 是什么？

`std::latch`（一次性倒计数门闩）、`std::barrier`（可复用屏障）、`counting_semaphore` / `binary_semaphore`，补充了原有同步原语。

2.7 异常、现代特性与工程实践

Q107. 异常安全保证有哪几级？

基本保证（不泄漏、对象仍有效）、强保证（失败则状态回滚）、不抛保证（`noexcept`）。实现手段：RAII、先做可能失败的工作再提交（copy-and-swap）。

Q108. noexcept 的作用？

声明函数不抛异常，使编译器可优化并让标准库选择更快路径（移动）。从 noexcept 函数抛出异常会调用 `std::terminate`。析构函数默认隐含 noexcept。

Q109. 析构函数中抛异常会怎样？

栈展开过程中析构函数再抛出未捕获的异常会导致 `std::terminate`。析构函数应吞下或记录错误，不要让异常逃出。

Q110. 异常与错误码如何取舍？

异常适合"不应经常发生"的错误并保持正常路径整洁，但有二进制体积和抛出时的性能开销；游戏、嵌入式和低延迟系统常禁用异常，使用错误码或 `std::expected`（C++23）。

Q111. 常见的未定义行为有哪些？

越界访问、空指针/悬空解引用、有符号溢出、违反严格别名、数据竞争、修改 const 对象、使用已销毁对象、违反 ODR、不匹配的 new/delete、虚函数调用时对象未完全构造、移位越界。使用 ASan/UBSan/TSan 与 `-Wall -Wextra` 排查。

Q112. C++20 模块（modules）解决什么问题？

用 `import` 取代头文件文本包含，加速编译、消除宏污染与 ODR 隐患，并控制导出接口（`export`）。工具链支持逐渐成熟，但项目迁移成本仍较高。

Q113. C++23 有哪些值得关注的特性？

`std::expected`、`std::print/println`、`std::mdspan`、`std::flat_map/flat_set`、deducing this、`std::generator`、`std::move_only_function`、`if consteval`、`std::stacktrace`、多维下标 `operator[]`。

Q114. 如何做 C++ 性能优化？

先测量（perf、VTune、benchmark）；改善数据布局（连续存储、AoS→SoA）与缓存局部性；避免不必要的拷贝和堆分配（`reserve`、`string_view`、移动）；减少虚调用与锁争用；使用合适的算法与容器；编译选项 `-O2/-O3`、LTO、PGO；向量化。

Q115. 如何设计线程池？

任务队列（互斥量 + 条件变量，或无锁队列）、固定数量工作线程、`submit` 返回 future、优雅停止（置标志并唤醒所有线程）、异常传播（捕获并写入 promise）、避免在池内线程阻塞等待同一池的任务以防饥饿。

Q116. 如何用 CMake 组织一个 C++ 项目？

使用现代 CMake：`target_link_libraries`、`target_include_directories`（PUBLIC/PRIVATE/INTERFACE）、`target_compile_features(cxx_std_20)`、`FetchContent` / `find_package`/vcpkg/Conan 管理依赖、`enable_testing()` + GoogleTest/Catch2、sanitizers 和 clang-tidy 集成，CI 中多编译器多平台构建。

Q117. 常见的 C++ 代码规范与静态检查工具？

C++ Core Guidelines、Google C++ Style；工具：clang-tidy、clang-format、cppcheck、AddressSanitizer 系列、include-what-you-use。强调 RAII、`const` 正确性、避免裸指针所有权、避免宏。

第三部分：Java

3.1 语言基础

Q1. JDK、JRE、JVM 的关系？

JVM 是执行字节码的虚拟机；JRE = JVM + 核心类库（运行环境）；JDK = JRE + 开发工具（javac、jar、jshell、jcmd 等）。从 JDK 11 起不再单独提供 JRE 发行版，可用 `jlink` 裁剪出自定义运行时。

Q2. Java 的基本数据类型有哪些？包装类有什么缓存？

8 种：byte、short、int、long、float、double、char、boolean。包装类 `Integer.valueOf` 缓存 -128~127（`IntegerCache`，上限可通过参数调整），`Long`、`Short`、`Byte`、`Character`（0~127）也有缓存，`Boolean` 复用 TRUE/FALSE。所以 `Integer a=127,b=127; a==b` 为 true，而 128 为 false。

Q3. == 与 equals 的区别？

`==` 对基本类型比较值，对引用类型比较地址；`equals` 默认实现也是比较地址，需要按业务重写。包装类、String 重写后比较内容。

Q4. equals 与 hashCode 的约定？

equals 为真则 hashCode 必须相等；hashCode 相等 equals 不一定为真；equals 需满足自反、对称、传递、一致，且对 null 返回 false。重写 equals 必须重写 hashCode，否则放入 HashMap/HashSet 会出错。

Q5. String 为什么不可变？有什么好处？

内部是 `final` 的 `byte[]`（JDK 9 起紧凑字符串，Latin1/UTF16），类为 `final` 且没有修改方法。好处：线程安全、可缓存 hashCode、可作 HashMap 键、字符串常量池复用、安全（类加载、网络参数不会被篡改）。

Q6. 字符串常量池与 intern 是什么？

字面量与 `intern()` 的结果存放于常量池（JDK 7 起位于堆中）。`new String("a")` 会创建新对象（并确保池中存在 "a"），`s.intern()` 返回池中对应对象。编译期常量拼接 `"a" + "b"` 会被折叠为 "ab"。

Q7. String、StringBuilder、StringBuffer 的区别？

String 不可变；StringBuilder 可变、非线程安全、性能高；StringBuffer 可变、方法加了 synchronized。循环拼接应使用 StringBuilder；JDK 9+ 的单行 `+` 拼接通过 invokedynamic 优化，但循环内仍会产生大量对象。

Q8. Java 是值传递还是引用传递？

始终是值传递。传对象时传递的是引用的副本：函数内通过该引用修改对象内容，调用者可见；但把形参指向新对象不影响调用者的变量。

Q9. final、finally、finalize 的区别？

`final`：修饰类（不可继承）、方法（不可重写）、变量（不可重新赋值；引用不可变但对象可变）。`finally`：try 块的收尾，几乎总会执行（除 `System.exit`、JVM 崩溃、线程被杀等）。`finalize`：对象回收前的钩子，已被废弃（JDK 9），应使用 `try-with-resources` 或 `Cleaner`。

Q10. 自动装箱与拆箱有哪些坑？

循环中频繁装箱产生大量对象；对 null 的包装类拆箱抛 NPE（如 `Integer x = map.get(k); int y = x;`）；三元表达式 `cond ? 1 : null` 类型提升可能触发拆箱；包装类比较要用 equals。

Q11. 抽象类和接口的区别？

抽象类：可有构造器、实例字段、非抽象方法，单继承；接口：多实现，字段隐含 `public static final`，Java 8 起可有 `default` / `static` 方法，Java 9 起可有 `private` 方法。接口表达能力（can-do），抽象类表达"是什么"并共享状态。

Q12. 重载与重写的区别？

重载（overload）：同类中方法名相同、参数不同，编译期根据静态类型绑定；重写（override）：子类覆盖父类方法，签名相同、返回类型协变、访问权限不能更小、不能抛更宽的受检异常，运行期动态绑定。

Q13. 静态内部类、成员内部类、局部类、匿名内部类的区别？

静态嵌套类不持有外部类引用；成员内部类隐式持有外部类实例（可能引起内存泄漏）；匿名内部类/lambda 在方法内定义，只能捕获 effectively final 局部变量。lambda 通过 invokedynamic 实现，不生成额外的类文件，且 `this` 指向外部类。

Q14. 泛型的类型擦除是什么？有什么限制？

编译后泛型信息被擦除为上界（通常 Object），编译器插入强制转换。限制：不能 `new T()`、`new T[]`、不能用基本类型作类型参数、不能对泛型类型 `instanceof List<String>`、静态成员不能引用类型参数、不能创建泛型异常。运行期可通过子类的 `getGenericSuperclass()` 获取（TypeToken 技巧）。

Q15. 通配符 `? extends` 与 `? super` 以及 PECS？

Producer Extends, Consumer Super：只读取用 `List<? extends T>`（不能写入，除 null），只写入用 `List<? super T>`（读取得到 Object）。例：`Collections.copy(List<? super T> dest, List<? extends T> src)`。

Q16. 反射是什么？有什么代价？

运行期通过 `Class`、`Method`、`Field`、`Constructor` 检查和操作类。用途：框架（Spring、Jackson、MyBatis）、动态代理、注解处理。代价：性能比直接调用差（现代 JIT 与 MethodHandle 已缩小差距）、破坏封装、编译期不安全；JDK 9+ 模块系统对 `setAccessible` 有限制，需要 `--add-opens`。

Q17. 注解是怎么工作的？

注解本身只是元数据，通过元注解 `@Retention`（SOURCE/CLASS/RUNTIME）、`@Target` 声明；运行期通过反射读取，或

在编译期通过注解处理器（APT，如 Lombok、MapStruct）生成代码。

Q18. 序列化中 transient 和 serialVersionUID 的作用？

`transient` 字段不参与默认序列化；`serialVersionUID` 用于版本校验，不一致会抛 `InvalidClassException`。JDK 原生序列化存在安全风险（反序列化漏洞）、性能与兼容问题，通常改用 JSON、Protobuf、Avro 等。

Q19. 浅克隆与深克隆？

`Object.clone()` 默认浅拷贝（需实现 `Cloneable`）；深拷贝需递归复制引用字段，或通过序列化、拷贝构造器、工厂方法。`clone` 设计有缺陷，推荐用拷贝构造器。

Q20. Object 类有哪些方法？

`equals`、`hashCode`、`toString`、`getClass`、`clone`、`finalize`、`wait` / `notify` / `notifyAll`。后三者必须在持有对象监视器时调用。

Q21. 枚举为什么适合做单例？

枚举由 JVM 保证只实例化一次，天然线程安全，且防止反射与反序列化破坏。《Effective Java》推荐 `enum Singleton { INSTANCE; }`。

Q22. record、sealed、text block 等新语法是什么？

Record（Java 16）：不可变数据载体，自动生成构造器、访问器、equals/hashCode/toString。Sealed 类（Java 17）：`sealed ... permits` 限制继承层次，配合模式匹配实现穷尽检查。文本块（Java 15）：`"""` 多行字符串。`var`（Java 10）局部变量类型推断。Switch 表达式（Java 14）与模式匹配 `instanceof`（Java 16）、switch 模式匹配（Java 21）。

Q23. Optional 应该怎么用？

作为"可能无结果"的方法返回值类型，避免返回 null；使用 `map` / `flatMap` / `orElse` / `orElseGet` / `orElseThrow`。不要用于字段、方法参数、集合元素；`orElse` 的参数总会被求值，昂贵计算用 `orElseGet`。

Q24. Stream API 的特点与注意事项？

惰性求值（中间操作 filter/map 到终端操作才执行）、可并行（`parallelStream` 使用公共 ForkJoinPool）、流只能消费一次。注意：不要在流中修改外部状态；并行流对小数据或阻塞 IO 不一定更快；`collect(Collectors.toMap)` 遇到重复键会抛异常；JDK 16 有 `toList()`。

Q25. switch 支持哪些类型？新写法有何不同？

byte/short/char/int 及包装类、String、enum；Java 21 起支持模式匹配与 `case null`。表达式形式 `case X -> value` 无贯穿（fall-through），可直接返回值，配合 `yield`。

3.2 集合框架

Q26. ArrayList 与 LinkedList 的区别？

ArrayList 基于动态数组（扩容 1.5 倍），随机访问 O(1)，尾部添加摊还 O(1)，中间插入删除需移动；LinkedList 是双向链表，头尾操作 O(1)，随机访问 O(n)，节点内存开销大、缓存不友好。实践中几乎总选 ArrayList（队列场景用 `ArrayDeque`）。

Q27. HashMap 的底层原理？

JDK 8：数组 + 链表 + 红黑树。`hash = h ^ (h >>> 16)` 扰动后与 `(n-1)` 取与得到桶下标；默认容量 16，负载因子 0.75，超过阈值时扩容为 2 倍并重新分布（利用高位判断元素留在原位或移动 `oldCap`）；链表长度≥8 且数组长度≥64 时树化，≤6 时退化回链表。允许一个 null 键。

Q28. 为什么 HashMap 容量必须是 2 的幂？

便于用位与代替取模（`(n-1) & hash`），并让扩容时元素要么留在原位置要么移动到"原位置 + oldCap"。传入非 2 的幂的初始容量时会被调整为不小于它的 2 的幂。

Q29. HashMap 为什么线程不安全？

并发 put 可能丢失数据；JDK 7 扩容时头插法在并发下可能形成环形链表导致 CPU 100%；JDK 8 改为尾插法避免了环，但仍有覆盖和 size 不一致问题。并发场景用 `ConcurrentHashMap`。

Q30. ConcurrentHashMap 的实现原理？

JDK 7：分段锁（Segment）。JDK 8：取消分段，使用 CAS 写入空桶，非空桶对头节点加 `synchronized`，链表过长时同样树化；size 通过 `baseCount` + `CounterCell` 分散计数（类似 LongAdder）。不允许 null 键值；`compute/merge` 等方法是原子的，但复合操作（check-then-act）仍需使用这些方法而非手写。

Q31. HashMap、Hashtable、TreeMap、LinkedHashMap 的区别？

HashMap：无序、非线程安全；Hashtable：方法全部 synchronized，已过时；TreeMap：红黑树，按键排序 O(log n)；LinkedHashMap：维持插入顺序或访问顺序。

Q32. 如何用 LinkedHashMap 实现 LRU 缓存？

```
          class LRU<K,V> extends LinkedHashMap<K,V> {
              private final int cap;
              LRU(int cap){ super(16, 0.75f, true); this.cap = cap; }   // acces
sOrder=true
              @Override protected boolean removeEldestEntry(Map.Entry<K,V> e){ r
eturn size() > cap; }
          }
```

并发场景可使用 Caffeine。

Q33. HashSet 的实现及判重原理？

内部是 HashMap，元素作为键、一个固定的 `PRESENT` 对象作为值；判重依赖 hashCode 与 equals。

Q34. fail-fast 与 fail-safe 是什么？

fail-fast：迭代期间检测到结构被修改（`modCount` 变化）抛 `ConcurrentModificationException`（ArrayList、HashMap）；fail-safe/弱一致：基于快照或弱一致遍历，不抛异常（CopyOnWriteArrayList、ConcurrentHashMap）。遍历时删除请用 `Iterator.remove()` 或 `removeIf`。

Q35. CopyOnWriteArrayList 的适用场景？

读多写少：写时复制整个数组并替换引用，读无锁。代价是写开销大、内存翻倍、读到的可能是旧数据。适合监听器列表、配置白名单。

Q36. PriorityQueue 与 ArrayDeque 如何使用？

`PriorityQueue` 是基于数组的二叉堆，默认小顶堆，`offer/poll` O(log n)，遍历不保证顺序；`ArrayDeque` 是循环数组实现的双端队列，可替代 `Stack` 与 `LinkedList` 做栈/队列，不允许 null。

Q37. Comparable 与 Comparator 的区别？

Comparable 是类自身的自然顺序（`compareTo`）；Comparator 是外部比较器，可有多种排序规则，如 `Comparator.comparing(Person::age).thenComparing(Person::name)`。注意不要用减法比较整数以防溢出，应使用 `Integer.compare`。

Q38. Arrays.asList 与 List.of 有什么陷阱？

`Arrays.asList` 返回固定大小、写穿透底层数组的列表，`add/remove` 抛 `UnsupportedOperationException`；`List.of`（Java 9）返回不可变列表，不允许 null 元素。需要可变副本时用 `new ArrayList<>(...)`。

Q39. Collections.unmodifiableList 与 List.copyOf 的区别？

前者是对原列表的只读视图，原列表变化会反映出来；后者是不可变的拷贝。

Q40. TreeMap/TreeSet 的实现与注意点？

红黑树，要求键可比较（Comparable 或传入 Comparator），并且比较器必须与 equals 一致，否则会出现"重复键"被当作相同的情况。提供 `floorKey`、`ceilingKey`、`subMap` 等导航方法。

Q41. WeakHashMap 的作用？

键为弱引用，键对象不再被强引用时，条目会被自动清除，适合缓存与元数据映射。注意值若强引用键会阻止回收。

3.3 异常处理

Q42. 受检异常与非受检异常的区别？

受检异常（继承 Exception 但非 RuntimeException）必须捕获或声明；非受检异常（RuntimeException 及子类、Error）不强制处理。Error（OOM、StackOverflowError）一般不应捕获处理。业务代码倾向于使用非受检异常加统一处理。

Q43. try-with-resources 是什么？

实现 `AutoCloseable` 的资源在块结束时自动按声明相反顺序关闭，并将关闭时的异常作为 suppressed 异常附加到主异常上。取代了冗长且易错的 try/finally。

Q44. finally 中的 return 会怎样？

finally 中的 return 会覆盖 try/catch 里的返回值甚至吞掉异常，应避免。try 里 `return x` 时，返回值在执行 finally 前已被暂存，finally 中修改基本类型变量不影响返回值，但修改返回的对象内容会影响。

Q45. 异常处理的最佳实践？

捕获具体异常、不要吞异常、保留原因链（`new XxxException(msg, cause)`）、尽早抛出晚捕获、不要用异常控制流程、日志只记录一次、在边界（Controller 全局异常处理器）统一转换。

Q46. NoClassDefFoundError 与 ClassNotFoundException 的区别？

`ClassNotFoundException` 是显式加载类（`Class.forName`）时找不到，受检异常；`NoClassDefFoundError` 是编译时存在但运行时缺失（或静态初始化失败后再次使用），属于 Error。

3.4 JVM 与内存

Q47. JVM 运行时数据区有哪些？

线程私有：程序计数器、Java 虚拟机栈、本地方法栈；线程共享：堆、方法区（JDK 8 起实现为元空间 Metaspace，位于本地内存）。另有直接内存（NIO）、运行时常量池。

Q48. 堆的分代结构及对象晋升？

新生代（Eden + S0 + S1，默认比例 8:1:1）和老年代。新对象在 Eden 分配，Minor GC 后存活对象复制到 Survivor，年龄增加到阈值（默认 15，可动态调整）或 Survivor 放不下则晋升老年代；大对象可直接进入老年代。G1/ZGC 采用区域化堆（Region），分代边界是逻辑的。

Q49. 对象创建过程是怎样的？

类加载检查 → 分配内存（指针碰撞或空闲列表，TLAB 线程本地缓冲避免竞争）→ 内存清零 → 设置对象头 → 执行构造方法。

Q50. 对象的内存布局是怎样的？

对象头（Mark Word + 类型指针，开启压缩指针时共 12 字节）+ 实例数据 + 对齐填充（8 字节对齐）。Mark Word 保存哈希码、GC 分代年龄、锁状态等。

Q51. 强、软、弱、虚引用的区别？

强引用不会被回收；软引用在内存不足时回收（缓存）；弱引用下次 GC 就会回收（WeakHashMap、ThreadLocal 的 key）；虚引用无法获取对象，仅用于回收通知（配合 ReferenceQueue、`Cleaner`）。

Q52. 如何判断对象可回收？GC Roots 有哪些？

Java 使用可达性分析，而非引用计数（避免循环引用问题）。GC Roots 包括：虚拟机栈中局部变量引用、静态变量、常量池引用、JNI 引用、活动线程、同步锁持有对象等。

Q53. 常见的垃圾回收算法？

标记-清除（碎片）、标记-复制（新生代，浪费空间）、标记-整理（老年代，移动开销）、分代收集。现代收集器还使用三色标记 + 写屏障/读屏障进行并发标记。

Q54. 主要的垃圾收集器及特点？

Serial（单线程）、Parallel（吞吐量优先，JDK 8 默认）、CMS（低停顿，JDK 9 废弃，JDK 14 移除）、G1（区域化、可预测停顿，JDK 9 起默认）、ZGC（染色指针/读屏障，亚毫秒级停顿，JDK 15 生产可用，JDK 21 引入分代 ZGC，JDK 23 默认分代）、Shenandoah（并发整理，OpenJDK）。

Q55. Minor GC、Major GC、Full GC 的区别？

Minor GC：只回收新生代；Major GC：通常指老年代回收（概念并不统一）；Full GC：整个堆（含元空间）。频繁 Full GC 常见原因：老年代空间不足、元空间不足、显式 `System.gc()`、晋升失败、内存泄漏。

Q56. G1 收集器的工作过程？

年轻代 GC（STW 复制）→ 并发标记周期（初始标记、并发标记、最终标记、清理）→ 混合回收（Young + 部分高垃圾占比的老年代 Region）。通过 `-XX:MaxGCPauseMillis` 设定目标停顿时间，Region 大小 1~32MB，超过半个 Region 的对象为 Humongous 对象。

Q57. 线上 OOM 如何排查？

1. 开启 `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=...`；2. 用 MAT/VisualVM/JProfiler 分析 dump，查看支配树（Dominator Tree）和 GC Roots 路径；3. 区分堆溢出、元空间、直接内存、栈

溢出、`unable to create native thread`；4. 配合 `jstat -gcutil`、`jmap -histo`、GC 日志（`-Xlog:gc*`）。

Q58. 类加载过程是怎样的？

加载 → 链接（验证、准备、解析）→ 初始化（执行 `<clinit>`，即静态变量赋值和静态块）→ 使用 → 卸载。准备阶段为静态变量设置零值，初始化阶段才执行赋值。触发初始化的时机包括 new、访问静态成员、反射、子类初始化等。

Q59. 什么是双亲委派模型？如何打破？

类加载器收到请求先委派给父加载器，父加载器无法加载才自己加载：Bootstrap → Platform（JDK 9 前为 Extension）→ Application → 自定义。好处：避免核心类被篡改、避免重复加载。打破场景：JDBC SPI（线程上下文类加载器）、Tomcat（Web 应用隔离）、OSGi、热部署。

Q60. JIT 编译与分层编译是什么？

HotSpot 先解释执行，统计热点方法/循环后由 C1（快速、少量优化）与 C2（深度优化）编译为机器码；分层编译（默认开启）结合两者。常见优化：方法内联、逃逸分析（标量替换、锁消除）、循环展开、去虚化。GraalVM 提供替代编译器与 AOT 原生镜像。

Q61. 逃逸分析有什么用？

分析对象是否逃出方法或线程：不逃逸则可能栈上分配（实际以标量替换实现）、锁消除、减少堆分配。

Q62. 常用 JVM 参数有哪些？

`-Xms/-Xmx`（堆初始/最大）、`-Xss`（线程栈）、`-XX:MetaspaceSize`、`-XX:MaxMetaspaceSize`、`-XX:+UseG1GC` / `-XX:+UseZGC`、`-XX:MaxGCPauseMillis`、`-XX:MaxDirectMemorySize`、`-Xlog:gc*`、`-XX:+HeapDumpOnOutOfMemoryError`。容器中建议 `-XX:MaxRAMPercentage`。

Q63. Java 中有哪些内存泄漏场景？

静态集合持续添加、未关闭的资源（连接、流）、监听器/回调未注销、ThreadLocal 在线程池中未 remove、缓存无界、内部类持有外部类、自定义类加载器无法卸载。

Q64. ThreadLocal 为什么可能内存泄漏？

`ThreadLocalMap` 的 key 是对 ThreadLocal 的弱引用，value 是强引用；线程池中的线程长期存活，key 被回收后 value 仍被 Thread 引用而无法回收。使用后务必在 finally 中 `remove()`。

Q65. 常用的 JVM 诊断工具？

`jps`、`jstack`（线程栈，找死锁）、`jmap`（堆 dump）、`jstat`（GC 统计）、`jcmd`、`jinfo`、JFR（Java Flight Recorder，低开销）、Arthas、async-profiler（火焰图）、VisualVM、MAT。

Q66. 栈溢出和堆溢出的区别？

StackOverflowError：线程栈深度不足（深递归）；OutOfMemoryError: Java heap space：堆无法再分配对象。其他 OOM：Metaspace、Direct buffer memory、GC overhead limit exceeded、unable to create new native thread。

Q67. 什么是 JMM（Java 内存模型）？

规定线程如何通过主内存与工作内存交互，保证可见性、有序性、原子性的规则。核心是 happens-before 关系：程序顺序、监视器锁、volatile 读写、线程启动/终止、传递性等。

3.5 并发编程

Q68. 创建线程有哪些方式？

继承 Thread、实现 Runnable、Callable + FutureTask、线程池 `ExecutorService`、`CompletableFuture`、虚拟线程（`Thread.ofVirtual()`、

`Executors.newVirtualThreadPerTaskExecutor()`）。推荐使用线程池或虚拟线程执行器，而不是直接 new Thread。

Q69. 线程有哪些状态？

NEW、RUNNABLE、BLOCKED（等待监视器锁）、WAITING（`wait()` / `join()` / `park()`）、TIMED_WAITING（带超时）、TERMINATED。

Q70. sleep 与 wait 的区别？

`sleep` 是 Thread 静态方法，不释放锁；`wait` 是 Object 方法，必须在 synchronized 内调用，会释放锁并等待 `notify`。`wait` 应在 while 循环中检查条件（虚假唤醒）。

Q71. synchronized 的原理与锁升级？

基于对象头的 Monitor（`monitorenter/monitorexit`，方法上为 ACC_SYNCHRONIZED 标志）。HotSpot 的锁状态：无锁 → 轻量级锁（CAS 自旋）→ 重量级锁（操作系统互斥量）；偏向锁在 JDK 15 起默认禁用并废弃，后续版本已移除。synchronized 可重入，且 JDK 6 后性能大幅改善。

Q72. volatile 保证什么？不保证什么？

保证可见性（写后立即对其他线程可见）和禁止指令重排序（内存屏障），不保证复合操作的原子性（`i++`）。典型用途：状态标志、双重检查锁中的单例引用。

Q73. 双重检查锁单例为什么要 volatile？

`instance = new Singleton()` 包含分配内存、初始化、赋引用三步，可能被重排为先赋引用后初始化，其他线程可能拿到未初始化完成的对象；volatile 禁止该重排。也可用静态内部类持有器或枚举实现单例。

Q74. CAS 是什么？ABA 问题怎么解决？

Compare-And-Swap：当值等于期望值时原子地更新，由 CPU 指令（`cmpxchg`）支持，Java 通过 `Unsafe` / `VarHandle` 暴露。ABA：值由

A→B→A 被误认为未变，可用 `AtomicStampedReference`（版本号）解决。CAS 的缺点：自旋开销、只能保证单变量原子。

Q75. AtomicInteger 与 LongAdder 的区别？

AtomicInteger 是单变量 CAS，高竞争下大量自旋失败；LongAdder 将计数分散到多个 Cell，求和时累加，写吞吐更高但读取（sum）不是精确快照，适合统计计数。

Q76. AQS 是什么？

AbstractQueuedSynchronizer：基于 volatile `state` 和 CLH 变体的 FIFO 等待队列，提供独占/共享获取与释放的框架，`ReentrantLock`、`Semaphore`、`CountDownLatch`、`ReentrantReadWriteLock` 等均基于它实现。线程入队后通过 `LockSupport.park` 阻塞。

Q77. ReentrantLock 与 synchronized 的区别？

ReentrantLock 是 API 级锁：可中断（`lockInterruptibly`）、可超时（`tryLock`）、可公平/非公平、可绑定多个 Condition；必须在 finally 中 unlock。synchronized 是语言关键字，自动释放、JVM 优化更好。两者都可重入。

Q78. 读写锁与 StampedLock？

`ReentrantReadWriteLock`：读读共享、读写互斥、写写互斥，写饥饿问题；`StampedLock` 提供乐观读（`tryOptimisticRead` + `validate`），读多写少时性能更好，但不可重入、不支持 Condition。

Q79. 线程池的核心参数及工作流程？

参数：corePoolSize、maximumPoolSize、keepAliveTime、unit、workQueue、threadFactory、handler。流程：线程数<core 则创建线程；否则入队；队列满且线程数<max 则创建非核心线程；否则执行拒绝策略。

Q80. 线程池拒绝策略有哪些？

AbortPolicy（抛 RejectedExecutionException，默认）、CallerRunsPolicy（调用者线程执行，起到限流作用）、DiscardPolicy（静默丢弃）、DiscardOldestPolicy（丢弃队列头）。也可自定义。

Q81. 为什么不推荐使用 Executors 创建线程池？

`newFixedThreadPool` / `newSingleThreadExecutor` 使用无界队列（可能 OOM），`newCachedThreadPool` / `newScheduledThreadPool` 的最大线程数几乎无上限。应直接使用 `ThreadPoolExecutor`，设定有界队列、命名线程工厂和拒绝策略。

Q82. 线程池大小如何设定？

CPU 密集型约 N+1（N 为核数）；IO 密集型约 N×(1+等待时间/计算时间)，或根据压测调整。要结合队列长度、下游承载能力和监控（活跃线程、队列堆积）。

Q83. CountDownLatch、CyclicBarrier、Semaphore 的区别？

CountDownLatch：一次性，计数归零后放行等待线程；CyclicBarrier：可重复使用，多个线程到达屏障后一起继续；Semaphore：控制同时访问资源的许可数（限流）。`Phaser` 是更灵活的分阶段同步器。

Q84. CompletableFuture 如何使用？

支持异步编排：`supplyAsync`、`thenApply`、`thenCompose`（扁平化）、`thenCombine`、`allOf` / `anyOf`、`exceptionally` / `handle`。注意指定自定义线程池，避免占用公共 ForkJoinPool。

Q85. 什么是死锁？如何排查与避免？

多个线程互相持有对方需要的锁。排查：`jstack` 会提示 "Found one Java-level deadlock"，或 JFR/Arthas 的 `thread -b`。避免：固定加锁顺序、使用 `tryLock` 超时、缩小锁范围、减少嵌套锁。

Q86. Java 21 虚拟线程是什么？适用场景？

由 JVM 调度的轻量级线程，映射到少量载体线程（carrier threads）上，阻塞时自动卸载，使"一请求一线程"模型可以支撑百万级并发。适合 IO 密集型任务；不适合 CPU 密集；避免在 synchronized 内长时间阻塞导致"固定（pinning）"（部分已在较新版本改进）；不要池化虚拟线程；结合 `ThreadLocal` 要注意内存。

Q87. 什么是结构化并发与作用域值？

`StructuredTaskScope`（预览特性）将一组子任务当作一个整体，生命周期受作用域约束，异常与取消可传播；`ScopedValue` 是 `ThreadLocal` 的更轻量不可变替代。以最新 JDK 版本的状态为准。

Q88. ForkJoinPool 与并行流的原理？

工作窃取（work-stealing）：每个工作线程有自己的双端队列，空闲线程从他人队列尾部窃取任务。`parallelStream` 默认使用公共池，阻塞任务会拖慢整个应用。

Q89. 如何保证多线程下的顺序执行？

`Thread.join`、`CountDownLatch`、`CompletableFuture` 链、单线程 Executor、`BlockingQueue` 传递、`Semaphore` / `Condition` 交替信号。

Q90. 生产者消费者模式在 Java 中怎么实现？

`BlockingQueue`（`ArrayBlockingQueue`、`LinkedBlockingQueue`、`SynchronousQueue`、`LinkedTransferQueue`、`DelayQueue`）；或 `wait/notify`、`ReentrantLock + Condition`。

3.6 IO 与网络

Q91. BIO、NIO、AIO 的区别？

BIO 同步阻塞，一连接一线程；NIO（`Selector` + `Channel` + `Buffer`）同步非阻塞，IO 多路复用（Linux 下 epoll）；AIO（NIO.2 的

`AsynchronousChannel`）异步，实际生产使用较少。Netty 基于 NIO 封装了 Reactor 线程模型。

Q92. 零拷贝是什么？Java 中如何体现？

减少数据在用户态与内核态之间的拷贝。`FileChannel.transferTo`（对应 sendfile）、`MappedByteBuffer`（mmap）、Netty 的 `CompositeByteBuf` 与 `FileRegion`。Kafka 使用 sendfile 提高消费吞吐。

Q93. 直接内存（DirectByteBuffer）的特点？

分配在堆外，避免 IO 时的堆内到本地内存拷贝；分配和回收成本高，由 Cleaner 在 GC 后释放，受 `-XX:MaxDirectMemorySize` 限制；泄漏时堆监控看不到。

Q94. Netty 的线程模型和关键组件？

主从 Reactor：BossGroup 接收连接，WorkerGroup 处理读写；`EventLoop` 绑定线程与 Selector；`ChannelPipeline` 由一串 `ChannelHandler` 组成，处理入站出站事件；`ByteBuf` 支持池化与引用计数。耗时业务应交给独立线程池。

3.7 Spring 与开发实践

Q95. Spring IoC 和 DI 是什么？

IoC（控制反转）：由容器负责创建对象并管理依赖；DI（依赖注入）：通过构造器、setter、字段注入依赖。推荐构造器注入（不可变、便于测试、依赖明确）。

Q96. Spring Bean 的生命周期？

实例化 → 属性填充 → Aware 回调（BeanNameAware 等）→ `BeanPostProcessor.postProcessBeforeInitialization` → `@PostConstruct` / `InitializingBean.afterPropertiesSet` / `init-method` → `postProcessAfterInitialization`（AOP 代理在此创建）→ 使用 → `@PreDestroy` / `DisposableBean`。

Q97. Bean 的作用域有哪些？

singleton（默认）、prototype、request、session、application、websocket。singleton 注入 prototype 时只注入一次，需用 `ObjectProvider` 或 `@Lookup`。Spring 的单例 Bean 默认不是线程安全的，注意可变状态。

Q98. Spring 如何解决循环依赖？

通过三级缓存提前暴露半成品 Bean：一级 singletonObjects（成品）、二级 earlySingletonObjects（早期引用）、三级 singletonFactories（ObjectFactory，必要时生成 AOP 代理）。仅限 singleton 的 setter/字段注入；构造器注入循环依赖无法解决。Spring Boot 2.6 起默认禁止循环依赖。

Q99. AOP 的实现原理？JDK 动态代理与 CGLIB 的区别？

运行期生成代理对象织入切面。JDK 动态代理基于接口（`Proxy` + `InvocationHandler`）；CGLIB 通过继承生成子类（不能代理 final 类/方法）。Spring Boot 默认优先使用 CGLIB（`proxyTargetClass=true`）。同类内部方法自调用不会经过代理。

Q100. Spring 事务的传播行为有哪些？

REQUIRED（默认，有则加入无则新建）、REQUIRES_NEW（挂起当前，新建）、NESTED（保存点嵌套）、SUPPORTS、NOT_SUPPORTED、MANDATORY、NEVER。

Q101. `@Transactional` 在哪些情况下会失效？

同类内部自调用、方法非 public（或被 final/private 修饰）、异常被 catch 吞掉、抛出受检异常而未配置 `rollbackFor`（默认只回滚 RuntimeException 和 Error）、数据库引擎不支持事务（如 MyISAM）、对象不是 Spring 管理的 Bean、传播行为配置不当、多线程中事务不跨线程。

Q102. Spring Boot 自动配置的原理？

`@SpringBootApplication` 包含 `@EnableAutoConfiguration`，通过 `AutoConfigurationImportSelector` 读取 `META-`

`INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`（Boot 2.7+/3.x，旧版为 spring.factories）中的配置类，再用 `@ConditionalOnClass`、`@ConditionalOnMissingBean`、`@ConditionalOnProperty` 等条件决定是否生效。Boot 3 要求 Java 17+，并迁移到 `jakarta.*` 命名空间。

Q103. Spring MVC 请求处理流程？

DispatcherServlet 接收请求 → HandlerMapping 找到 Handler（Controller 方法）→ HandlerAdapter 调用（参数解析、数据绑定、校验）→ 返回值处理（`@ResponseBody` 通过 HttpMessageConverter 序列化）或 ViewResolver 渲染视图 → 异常由 HandlerExceptionResolver / `@ControllerAdvice` 处理。拦截器与过滤器的区别：Filter 属于 Servlet 规范，在 DispatcherServlet 之前；Interceptor 属于 Spring MVC。

Q104. MyBatis 中 `#{}` 与 `${}` 的区别？

`#{}` 使用预编译参数占位符（`?`），防止 SQL 注入；`${}` 是字符串直接拼接，仅用于动态表名/列名等无法参数化的场景，且必须白名单校验。

Q105. ORM 中的 N+1 问题是什么？怎么解决？

查询 N 个主对象后，每个对象再触发一次关联查询，共 N+1 条 SQL。解决：`JOIN FETCH`、`@EntityGraph`、批量抓取（`@BatchSize`）、二次 IN 查询、DTO 投影。

Q106. 如何设计接口幂等？

唯一业务键 + 数据库唯一索引、去重表、请求 ID/Token（先获取后一次性使用，Redis `SETNX`）、状态机（只允许合法状态流转）、乐观锁版本号。支付、下单、消息消费都要考虑重试导致的重复。

Q107. 数据库连接池（HikariCP）关键参数与排查？

`maximumPoolSize`、`minimumIdle`、`connectionTimeout`、`maxLifetime`（应小于数据库/代理的连接超时）、`idleTimeout`、`leakDetectionThreshold`。连接池耗尽常见原因：慢 SQL、事务过长、连接未归还、线程池过大。

Q108. SLF4J 与日志框架如何选择？

SLF4J 是门面，Logback/Log4j2 是实现。使用参数化日志 `log.info("x={}", x)`；注意日志级别、MDC 传递 traceId、异步日志、不要在日志中输出敏感数据。曾出现的 Log4Shell（CVE-2021-44228）提醒要及时升级依赖。

Q109. Maven 依赖冲突如何排查？

`mvn dependency:tree` 查看依赖树，利用"最近路径优先、先声明优先"规则；用 `<exclusions>` 排除，`dependencyManagement` / BOM 统一版本，`maven-enforcer-plugin` 检查冲突。Gradle 使用 `dependencies` 与 `dependencyInsight`。

Q110. 单元测试怎么写？JUnit 与 Mockito 的基本用法？

JUnit 5：`@Test`、`@BeforeEach`、`@ParameterizedTest`、`assertThrows`；Mockito：`@Mock`、`when(...).thenReturn(...)`、`verify(...)`、`ArgumentCaptor`。集成测试使用 `@SpringBootTest` 或 Testcontainers 启动真实依赖。目标是测行为而非实现，减少过度 mock。

Q111. 常见设计模式及 Java 中的例子？

单例（Runtime、枚举）、工厂（`Calendar.getInstance`）、建造者（`StringBuilder`、Lombok `@Builder`）、代理（Spring AOP）、装饰器（`BufferedInputStream`）、适配器（`InputStreamReader`）、观察者（事件监听）、策略（`Comparator`）、模板方法（`AbstractList`、`JdbcTemplate`）、责任链（Servlet Filter）。

Q112. 微服务相关：限流、熔断、降级的区别？

限流：控制入口流量（令牌桶/漏桶/滑动窗口）；熔断：调用下游失败率过高时暂时断开快速失败，半开探测恢复（Resilience4j、Sentinel）；降级：主功能不可用时返回兜底结果或关闭次要功能。同时需要超时与重试（配合退避和幂等）。

Q113. 如何定位 Java 服务 CPU 飙高？

`top -Hp <pid>` 找到高 CPU 线程，转为十六进制，在 `jstack` 输出中按 nid 定位线程栈；或使用 Arthas `thread -n 3`、async-profiler 火焰图。常见原因：死循环、频繁 Full GC、正则回溯、序列化/锁竞争。

Q114. Java 的 SPI 机制是什么？

Service Provider Interface：接口定义在核心库，实现由第三方放在 `META-INF/services/接口全名` 中，通过 `ServiceLoader.load` 在运行期发现，用于 JDBC 驱动、日志门面等。Spring 有自己的 `SpringFactoriesLoader`。

Q115. Java 模块系统（JPMS）是什么？

JDK 9 引入，用 `module-info.java` 声明 `requires` / `exports`，实现强封装和可靠的依赖。应用层使用不如 Maven/Gradle 普遍，但 JDK 本身已模块化，`jlink` 可生成最小运行时镜像。

第四部分：Python

4.1 语言基础

Q1. Python 是怎样的语言？解释型还是编译型？

动态强类型、多范式、自动内存管理。CPython 先把源码编译为字节码（`.pyc`，存于 `__pycache__`），再由虚拟机解释执行。动态指变量无固定类型；强类型指不会隐式转换不兼容的类型（`"1" + 1` 抛 `TypeError`）。

Q2. 可变对象与不可变对象有哪些？

不可变：int、float、bool、str、bytes、tuple、frozenset、None；可变：list、dict、set、bytearray、自定义对象。不可变对象可哈希（元素也不可变时的 tuple 可哈希），可作字典键。

Q3. is 和 == 的区别？

`==` 比较值（调用 `__eq__`），`is` 比较对象身份（`id`）。判断 None 用 `is None`。小整数（-5~256）和部分短字符串被驻留，所以 `a is b` 的结果不可依赖。

Q4. 浅拷贝与深拷贝？

`copy.copy`、`list(x)`、`x[:]`、`dict.copy()` 为浅拷贝，只复制外层容器，内部对象共享；`copy.deepcopy` 递归复制并用 memo 处理循环引用。自定义类可实现 `__copy__` / `__deepcopy__`。

Q5. Python 的参数传递是值传递还是引用传递？

传的是对象引用（"按对象共享/call by sharing"）：函数内重新绑定形参不影响外部，但原地修改可变对象会影响外部。

Q6. 默认参数为可变对象会有什么问题？

默认值在函数定义时只求值一次：

```
def f(x, lst=[]):
    lst.append(x); return lst
f(1); f(2)   # [1, 2] —— 共享同一个列表
```

正确做法：`lst=None`，函数内 `if lst is None: lst = []`。

Q7. *args 和 **kwargs 是什么？

`*args` 收集多余的位置参数为元组，`**kwargs` 收集多余关键字参数为字典；调用时 `f(*seq, **mapping)` 可解包。参数顺序：位置、`*args`、关键字专用参数、`**kwargs`；`/` 前为仅位置参数（3.8），`*` 后为仅关键字参数。

Q8. LEGB 作用域规则是什么？

名称查找顺序：Local → Enclosing（外层函数）→ Global → Built-in。赋值默认创建局部变量；`global` 声明使用全局变量，`nonlocal` 声明使用外层函数变量。

Q9. 什么是闭包？常见陷阱？

内层函数引用了外层函数的变量并被返回。陷阱是延迟绑定：

```
fs = [lambda: i for i in range(3)]
[f() for f in fs]   # [2, 2, 2]；修复：lambda i=i: i
```

Q10. 装饰器是什么？如何写带参数的装饰器？

装饰器是接受函数并返回新函数的可调用对象，`@dec` 等价于 `f = dec(f)`。

```python
import functools
def retry(n):
    def deco(fn):
        @functools.wraps(fn)           # 保留元信息
        def wrapper(*a, **kw):
            for i in range(n):
                try: return fn(*a, **kw)
                except Exception:
                    if i == n - 1: raise
        return wrapper
    return deco
```

Q11. 生成器与 yield 是怎么工作的？

含 `yield` 的函数返回生成器对象，调用 `next()` 时执行到下一个 `yield` 暂停并保存状态，惰性产生值，节省内存。支持 `send()`、`throw()`、`close()`，`yield from` 委托子生成器。

Q12. 可迭代对象与迭代器的区别？

可迭代对象实现 `__iter__`，返回迭代器；迭代器实现 `__iter__`（返回自身）和 `__next__`，耗尽后抛 `StopIteration`。列表是可迭代对象但不是迭代器；迭代器只能遍历一次。

Q13. 列表推导式与生成器表达式的区别？

`[x*x for x in it]` 立即生成列表；`(x*x for x in it)` 惰性生成器，内存小。推导式有自己的作用域（Python 3），支持嵌套与条件，也有字典/集合推导式。

Q14. 上下文管理器与 with 语句是什么？

实现 `__enter__`/`__exit__` 的对象，保证资源释放（文件、锁、连接）。更简单的写法：

```
from contextlib import contextmanager
@contextmanager
def cd(path):
    old = os.getcwd(); os.chdir(path)
    try: yield
    finally: os.chdir(old)
```

`__exit__` 返回真值会吞掉异常。

Q15. lambda、map、filter、reduce 怎么用？推荐吗？

lambda 是单表达式匿名函数。`map` / `filter` 返回惰性迭代器；`functools.reduce` 做归约。大多数情况下推导式更可读；`sorted(key=lambda ...)` 是 lambda 的常见合理用法。

Q16. 海象运算符和 match 语句是什么？

海象 `:=`（3.8）在表达式内赋值：`if (n := len(a)) > 10:`。`match`（3.10）是结构化模式匹配，可匹配序列、映射、类、守卫：

```
match cmd:
    case ["go", direction]: ...
    case {"type": "move", "x": x}: ...
    case Point(x=0, y=0): ...
```

Q17. f-string 有哪些用法？

`f"{name!r:>10}"`、`f"{x=}"`（3.8 调试输出）、`f"{value:.2f}"`、嵌套表达式。3.12 放宽了引号与嵌套限制。注意不要用 f-string 构造 SQL，应使用参数化查询。

Q18. 解包与星号表达式？

`a, *rest = seq`；`first, *_, last = seq`；字典合并 `{**d1, **d2}`，3.9 起可用 `d1 | d2`。

Q19. Python 3.8 之后有哪些重要新语法特性？

3.8 海象、仅位置参数；3.9 字典合并运算符、内置泛型 `list[int]`；3.10 `match`、`X | Y` 类型联合、更好的报错；3.11 `ExceptionGroup` / `except*`、`tomllib`、显著提速；3.12 `type` 语句与泛型语法（PEP 695）、f-string 放宽；3.13 实验性 free-threaded 构建与 JIT、新 REPL；3.14 t-string、注解延迟求值、free-threaded 受官方支持。具体以所用版本文档为准。

Q20. `if __name__ == "__main__":` 的作用？

脚本直接执行时 `__name__` 为 `"__main__"`，被 import 时为模块名。用此判断避免在被导入时执行主逻辑。

4.2 面向对象

Q21. 类属性与实例属性的区别？

类属性属于类，所有实例共享；实例属性属于具体实例。读取时先查实例再查类；通过实例赋值会创建同名实例属性，不会修改类属性（可变类属性如列表被各实例共享是常见 bug）。

Q22. classmethod、staticmethod、property 的区别？

`@classmethod` 第一个参数是类（`cls`），常用于替代构造器；`@staticmethod` 无隐式参数，仅是放在类命名空间里的函数；`@property` 把方法变为属性访问，可配合 `setter` / `deleter` 做校验与惰性计算。

Q23. `__new__` 与 `__init__` 的区别？

`__new__` 是静态方法，负责创建并返回实例（可用于单例、不可变类型子类化）；`__init__` 初始化已创建的实例。先调用 `__new__`，返回的是该类实例才会调用 `__init__`。

Q24. 什么是 MRO？多继承如何解析？

方法解析顺序，Python 3 使用 C3 线性化，保证子类先于父类、保持声明顺序。可通过 `Cls.__mro__` 或 `Cls.mro()` 查看。菱形继承中每个基类只出现一次。

Q25. super() 做了什么？

按 MRO 顺序返回下一个类的代理，而不是简单的"父类"。多继承中协作调用 `super().__init__()` 保证每个类只初始化一次，所有类应使用一致签名（通常 `**kwargs` 传递）。

Q26. 常用魔术方法有哪些？

`__init__`、`__repr__`、`__str__`、`__len__`、`__getitem__/__setitem__`、`__iter__/__next__`、`__call__`、`__enter__/__exit__`、`__eq__/__lt__`、`__hash__`、`__add__`（运算符重载）、`__getattr__/__getattribute__/__setattr__`、`__del__`。

Q27. `__repr__` 与 `__str__` 的区别？

`__repr__` 面向开发者，应尽量能复现对象（明确无歧义）；`__str__` 面向用户的可读字符串。缺少 `__str__` 时回退到 `__repr__`。

Q28. 重写 `__eq__` 为什么还需要考虑 `__hash__`？

只定义 `__eq__` 会使类的 `__hash__` 被设为 None（不可哈希）。如果对象要放入集合或作字典键，需要同时定义一致的 `__hash__`（相等的对象必须有相同哈希），且依赖的字段应不可变。

Q29. `__slots__` 有什么作用？

声明固定属性集合，不再为每个实例创建 `__dict__`，节省内存、属性访问略快，但限制动态添加属性且与多继承、弱引用需额外处理。

Q30. 描述符协议是什么？

实现 `__get__`、`__set__`、`__delete__` 的对象，作为类属性时拦截实例的属性访问。`property`、方法绑定、`classmethod`、`staticmethod` 都是描述符。数据描述符（有 `__set__`）优先于实例 `__dict__`。

Q31. 属性查找顺序是什么？

`type(obj).__mro__` 中查找数据描述符 → 实例 `__dict__` → 非数据描述符/类属性 → `__getattr__`（兜底）。

Q32. 什么是元类？

类的类，默认是 `type`。`class A(metaclass=Meta)` 可在类创建时定制（注册子类、校验、自动添加方法）。实际中多数需求可用类装饰器或 `__init_subclass__` 实现，更简单。

Q33. 抽象基类（ABC）怎么用？

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): ...
```

含未实现抽象方法的类不能实例化。`typing.Protocol` 提供基于结构的静态"鸭子类型"。

Q34. dataclass 是什么？

`@dataclass` 自动生成 `__init__`、`__repr__`、`__eq__`（可选排序、`frozen=True` 不可变、`slots=True`，3.10）。字段默认值用 `field(default_factory=list)`，不能直接使用可变默认值。

Q35. 鸭子类型的含义？

"走起来像鸭子就是鸭子"：只关心对象是否有所需方法/行为，而非其类型。配合 EAFP（先做后处理异常）风格。

Q36. 如何实现单例？

常见方式：模块级变量（最 Pythonic）、`__new__` 中缓存实例、装饰器、元类。多线程创建需加锁。

Q37. `isinstance` 与 `type()` 的区别？

`isinstance(x, C)` 考虑继承（及 ABC 的虚拟子类），`type(x) is C` 只匹配确切类型。优先使用 `isinstance`。

Q38. `__getattr__` 与 `__getattribute__` 的区别？

`__getattribute__` 对所有属性访问调用；`__getattr__` 仅在常规查找失败后调用。重写前者需用 `super()` 避免无限递归。

4.3 数据结构与内置类型

Q39. list 与 tuple 的区别？

list 可变、动态数组，tuple 不可变、更省内存、可哈希（元素可哈希时）。tuple 用于固定结构的记录、字典键、函数多返回值。

Q40. list 的底层实现与时间复杂度？

动态数组（存放对象指针），过度分配以摊还 O(1) 的 `append`。索引 O(1)；`insert(0, x)`、`pop(0)`、`in`、`remove` 为 O(n)；切片复制 O(k)。需要双端队列用 `collections.deque`。

Q41. dict 的实现与有序性？

哈希表，开放寻址；3.6 起使用紧凑实现（索引表 + 条目数组），3.7 起语言保证按插入顺序。键需可哈希；平均 O(1) 查找，删除/重新插入会改变顺序。

Q42. set 的特点？

基于哈希表，元素唯一且可哈希，支持并交差对称差（`| & - ^`）；`in` 平均 O(1)。`frozenset` 不可变，可作字典键。

Q43. sort 与 sorted 的区别？用了什么算法？

`list.sort()` 原地排序返回 None，`sorted()` 返回新列表，可作用于任意可迭代对象。两者使用稳定的 Timsort（归并+插入，最坏 O(n log n)，近乎有序时接近 O(n)）。`key` 函数每个元素只调用一次。

Q44. collections 里有哪些常用数据结构？

`defaultdict`（缺省工厂）、`Counter`（计数，`most_common`）、`deque`（O(1) 双端操作）、`OrderedDict`（有序，支持 `move_to_end`）、`namedtuple`、

`ChainMap`。

Q45. heapq 和 bisect 如何使用？

`heapq` 是基于列表的最小堆：`heappush/heappop/heapify/nlargest/nsmallest`；最大堆可对数值取负。`bisect_left/right` 在有序序列上二分查找，`insort` 保持有序插入。

Q46. 切片有哪些细节？

`a[start:stop:step]` 左闭右开，越界不报错，`a[::-1]` 反转。切片总是返回新对象（浅拷贝）。切片赋值可替换、删除、插入。

Q47. 字典合并与常用方法？

`d1 | d2`（3.9）、`{**d1, **d2}`、`d.update(...)`。`get(k, default)`、`setdefault`、`pop(k, default)`、`items()` 视图。遍历时修改字典大小会抛 `RuntimeError`。

Q48. 字符串拼接如何高效？

循环中用 `''.join(parts)` 而不是 `+=`（CPython 对某些情况有就地优化，但不可依赖）；格式化用 f-string。

Q49. str 与 bytes 的区别？编码问题怎么处理？

str 是 Unicode 码点序列，bytes 是字节序列。`s.encode('utf-8')` / `b.decode('utf-8')`；打开文本文件应显式指定 `encoding='utf-8'`；处理错误用 `errors='replace'/'ignore'`。3.15 计划默认 UTF-8 模式（PEP 686），以文档为准。

Q50. 整数溢出问题？

Python int 是任意精度，不会溢出，但运算开销更大。`float` 是 IEEE 754 双精度；精确小数用 `decimal.Decimal` 或 `fractions.Fraction`。

Q51. `//`、`%` 对负数的行为？

`//` 向下取整（floor）：`-7 // 2 == -4`；`%` 结果与除数同号：`-7 % 3 == 2`，满足 `a == (a//b)*b + a%b`。

Q52. 如何高效去重并保持顺序？

`list(dict.fromkeys(seq))`（利用字典保序）。需要集合运算时用 set，但不保序。

Q53. enumerate、zip、any/all 的用法？

`for i, x in enumerate(seq, start=1)`；`zip(a, b)` 以最短为准，3.10 起 `strict=True` 可要求等长；`itertools.zip_longest` 补齐；`any/all` 短路求值，可传生成器。

Q54. itertools 与 functools 常用函数？

itertools：`chain`、`islice`、`groupby`（需先排序）、`product`、`permutations`、`combinations`、`accumulate`、`count`、`cycle`、`batched`（3.12）。functools：`lru_cache` / `cache`、`partial`、`reduce`、`wraps`、`singledispatch`、`cached_property`、`total_ordering`。

4.4 内存管理与并发

Q55. Python 的内存管理机制？

CPython 用引用计数为主，分代垃圾回收（3 代）处理循环引用；小对象由 pymalloc 的 arena/pool/block 管理，减少碎片与系统调用。`sys.getrefcount` 可查看计数，`gc` 模块可手动控制。

Q56. 循环引用如何处理？weakref 是什么？

引用计数无法回收循环引用，由 GC 检测不可达的循环并回收（带 `__del__` 的对象在 3.4 后也可回收）。`weakref` 持有不增加引用计数的引用，用于缓存、观察者，避免内存泄漏。

Q57. 什么是 GIL？为什么存在？

全局解释器锁：同一时刻只有一个线程执行 Python 字节码。它简化了 CPython 的内存管理（引用计数的线程安全）和 C 扩展集成。线程在 IO 阻塞时会释放 GIL，并且每隔一个切换间隔（默认 5ms）请求切换。

Q58. GIL 对多线程有什么影响？

CPU 密集型任务多线程无法利用多核（甚至更慢）；IO 密集型任务仍受益。绕过：多进程（`multiprocessing`、`concurrent.futures.ProcessPoolExecutor`）、C 扩展/NumPy（释放 GIL）、子解释器、异步 IO。

Q59. free-threaded（无 GIL）Python 的现状？

PEP 703 在 3.13 引入可选的 free-threaded 构建（实验性），3.14 起受官方支持但仍是可选构建；单线程性能和 C 扩展生态兼容仍在演进。3.12 的 PEP 684 让每个子解释器拥有独立 GIL。选型前应核查依赖库是否支持。

Q60. 线程、进程、协程如何选择？

IO 密集且并发量大：`asyncio`（协程）；IO 密集且库是阻塞式：线程池；CPU 密集：多进程或 C 扩展；需要隔离与容错：多进程。

Q61. threading 中常见同步原语？

`Lock`、`RLock`、`Condition`、`Event`、`Semaphore`、`Barrier`、`queue.Queue`（线程安全队列）。`with lock:` 保证释放。

Q62. dict、list 的操作是线程安全的吗？

单个字节码级操作（如 `list.append`、`dict[k] = v`）在 CPython 中由于 GIL 通常是原子的，但复合操作（`x += 1`、check-then-set）不是，仍需加锁。依赖 GIL 行为在 free-threaded 构建下不可靠。

Q63. multiprocessing 要注意什么？

进程间不共享内存，需通过 `Queue`、`Pipe`、共享内存（`shared_memory`）、Manager 通信；参数需可被 pickle；Windows/macOS 默认 spawn 启动方式，需要 `if __name__ == "__main__":` 保护；Linux 默认 fork（3.14 起默认改为 forkserver）。

Q64. asyncio 的基本概念？

单线程事件循环调度协程（`async def`）；`await` 让出控制权；`asyncio.run(main())` 启动；`create_task` 并发执行，`gather` / `TaskGroup`（3.11）等待多个任务，`wait_for` / `timeout` 设置超时。

Q65. asyncio 中如何处理阻塞调用？

阻塞函数会卡住整个事件循环，应使用 `await asyncio.to_thread(func)`（3.9）或 `loop.run_in_executor`，或换用异步库（`aiohttp`、`asyncpg`、`httpx.AsyncClient`）。CPU 密集用进程池。

Q66. 协程、任务、Future 的区别？

协程对象本身不会运行，需 `await` 或包装为 Task；Task 是调度协程的 Future 子类；Future 表示尚未完成的结果。未 await 的协程会触发 "never awaited" 警告。

Q67. 取消任务与超时如何处理？

`task.cancel()` 在任务的下一个 await 处抛出 `CancelledError`，应让其继续传播（不要吞掉）；`asyncio.timeout()`（3.11）上下文管理器。清理使用 `try/finally`。

Q68. 生成器和协程的关系？

协程最初由生成器的 `send` 和 `yield from` 演化而来；现代使用 `async/await`。`async for` / `async with` 配合异步迭代器与异步上下文管理器。

Q69. concurrent.futures 的用法？

`ThreadPoolExecutor` / `ProcessPoolExecutor`：`submit` 返回 Future，`map` 按序返回结果，`as_completed` 按完成顺序。用 `with` 管理生命周期。

4.5 模块、异常与类型

Q70. import 机制是怎样的？

先查 `sys.modules` 缓存，未找到则通过 `sys.meta_path` 上的查找器在 `sys.path` 中定位并执行模块代码。模块只在首次导入时执行一次。`importlib.reload` 可重载。

Q71. 如何避免循环导入？

重构公共部分到第三方模块、延迟导入（函数内 import）、依赖注入、仅类型检查时导入（`if TYPE_CHECKING:`）。

Q72. 包的 `__init__.py` 作用？相对导入与绝对导入？

标记包并可定义包级导出（`__all__`）。3.3 起支持无 `__init__.py` 的命名空间包。绝对导入 `from pkg.mod import x`；显式相对导入 `from .mod import x` 只能在包内使用。

Q73. 虚拟环境与依赖管理有哪些工具？

`venv`、`virtualenv`、`pip`、`pip-tools`、`poetry`、`pipenv`、`uv`（快速的 Rust 实现）、`conda`。现代项目用 `pyproject.toml` 声明依赖，锁文件保证可复现构建。

Q74. 异常处理的完整结构？

```
try: ...
except (ValueError, KeyError) as e: ...
except Exception: raise
else: ...      # 无异常时执行
finally: ...   # 总是执行
```

`raise ... from e` 保留异常链；`except*` 与 `ExceptionGroup`（3.11）处理多个并发异常。

Q75. EAFP 与 LBYL 的区别？

EAFP（请求原谅比许可更容易）：直接尝试并捕获异常，Pythonic；LBYL（三思而后行）：先判断条件再操作。EAFP 可避免检查与使用之间的竞态。

Q76. 如何自定义异常？

继承 `Exception`（不要继承 `BaseException`），建立业务异常层次，信息放入属性。不要使用裸 `except:`，它会捕获 `KeyboardInterrupt` 和 `SystemExit`。

Q77. 类型注解有什么用？

类型提示不影响运行，供 mypy/pyright 静态检查与 IDE 提示：`list[int]`、`dict[str, int]`、`X | None`、`Callable`、`TypeVar`、`Generic`、`Protocol`、`TypedDict`、`Literal`、`Final`、`Self`。可结合 pydantic 做运行期校验。

Q78. `typing.Optional[int]` 表示什么？

等价于 `int | None`，并不表示参数可省略（那需要默认值）。

Q79. pydantic 与 dataclass 的区别？

dataclass 只生成样板代码，不校验；pydantic 在运行期做解析/校验与类型转换，常用于 API 数据模型和配置（FastAPI）。

4.6 性能与工程实践

Q80. 如何分析 Python 程序的性能？

`cProfile` / `profile` + `pstats`、`line_profiler`、`py-spy`（采样，无需改代码）、`scalene`、`timeit`、`tracemalloc`（内存）、`memory_profiler`。先找热点，再优化。

Q81. 常见的 Python 性能优化手段？

选对数据结构（set/dict 查找）、用内置函数和推导式、避免重复属性查找、`lru_cache` 缓存、生成器减少内存、NumPy 向量化、批量 IO、多进程/异步、必要时 Cython、Numba、PyPy 或用 Rust/C 重写热点。

Q82. lru_cache 如何使用？

`@functools.lru_cache(maxsize=128)`，参数必须可哈希；`cache` 为无上限版本（3.9）；`cache_clear()` 清空。注意缓存会持有参数对象，方法上使用会持有 self。

Q83. 为什么 Python 比 C/Java 慢？CPython 近年如何提速？

动态类型、解释执行、对象开销、GIL。3.11 的"专用自适应解释器"（特化字节码）提速约 25%；3.13 起有实验性 copy-and-patch JIT；free-threaded 提升多核可扩展性。

Q84. 如何读取大文件？

逐行迭代 `for line in f`，分块 `f.read(size)`，`mmap`，pandas 的 `chunksize`，生成器管道。避免一次性 `read()`。

Q85. 常见的 Web 框架与 WSGI/ASGI？

WSGI 是同步接口标准（Django 传统模式、Flask，部署 gunicorn/uWSGI）；ASGI 是异步接口（FastAPI、Starlette、Django 3+，部署 uvicorn/hypercorn），支持 WebSocket。FastAPI 基于类型注解自动生成校验和 OpenAPI 文档。

Q86. Django 与 Flask 的区别？

Django 是"全家桶"（ORM、Admin、认证、迁移、模板）；Flask 是微框架，按需选择扩展，灵活。FastAPI 偏向高性能 API 与类型驱动。

Q87. ORM（Django ORM/SQLAlchemy）的常见问题？

N+1 查询（Django `select_related`/`prefetch_related`，SQLAlchemy `joinedload`/`selectinload`）、事务边界、连接池、迁移管理、原生 SQL 的参数化。

Q88. 如何写测试？pytest 的特点？

`pytest` 使用普通 `assert`、fixture（依赖注入）、参数化 `@pytest.mark.parametrize`、插件生态；mock 使用 `unittest.mock.patch` /

`pytest-mock`；覆盖率 `pytest-cov`；属性测试用 `hypothesis`。

Q89. logging 如何正确使用？

使用 `logging.getLogger(__name__)`，配置级别、格式和处理器；使用 `logger.info("x=%s", x)` 延迟格式化；不要用 `print` 做日志；异常用 `logger.exception`。结构化日志可使用 `structlog`。

Q90. 代码规范与静态检查工具？

PEP 8；格式化 `black` / `ruff format`；lint `ruff` / `flake8` / `pylint`；类型检查 `mypy` / `pyright`；导入排序 `isort`（ruff 也支持）；`pre-commit` 钩子统一执行。

Q91. 如何打包与发布 Python 项目？

使用 `pyproject.toml`（PEP 517/518/621），构建后端如 setuptools、hatchling、poetry-core；`python -m build` 生成 sdist 与 wheel，`twine upload` 发布到 PyPI。应用部署常用 Docker 镜像加锁定依赖。

Q92. 如何在生产中处理密钥和配置？

环境变量或密钥管理服务，不提交到仓库；使用 `pydantic-settings` 加载并校验；按环境分层配置；日志脱敏。

Q93. Python 如何调用 C/Rust 代码？

`ctypes`、`cffi`、C 扩展 API、Cython、pybind11、Rust 的 PyO3/maturin。用于性能热点或复用已有库。

Q94. `*` 和 `**` 在函数调用与定义中的各种含义？

定义时：收集参数（`*args`、`**kwargs`）；调用时：解包可迭代对象/映射；定义中单独的 `*` 表示之后为仅关键字参数。

Q95. 什么是 pickle？有何风险？

Python 对象序列化格式；反序列化不可信数据可执行任意代码，必须只加载可信来源。跨语言或对外交换用 JSON、MessagePack、Protobuf。

Q96. `__del__` 与资源释放的最佳实践？

`__del__` 调用时机不确定（循环引用、解释器退出），不要依赖它释放关键资源，使用 `with` 上下文管理器或显式 `close()`；可用 `weakref.finalize` 注册清理。

Q97. Python 中如何实现缓存/限流/重试？

内存缓存 `functools.cache`、`cachetools`（TTL/LRU）；分布式缓存 Redis；限流用令牌桶/滑动窗口（`asyncio.Semaphore`、`aiolimiter`）；重试使用 `tenacity`，需配合指数退避与幂等。

Q98. 常用的 PEP 有哪些值得知道？

PEP 8（风格）、PEP 20（Zen of Python）、PEP 257（docstring）、PEP 484（类型提示）、PEP 572（海象）、PEP 634（match）、PEP 703（free-threaded）、PEP 621（pyproject 元数据）。

Q99. 如何调试 Python 程序？

`pdb`/`breakpoint()`、IDE 调试器、`traceback`、`faulthandler`（段错误转储）、`logging` 分级、`sys.settrace`。生产环境用 `py-spy dump` 抓取线程栈。

Q100. 多重继承中的 Mixin 是什么？

提供单一功能、不独立使用的小类，放在基类前面组合行为（如日志、序列化）。Mixin 应避免携带状态、不要与具体基类有复杂耦合。

Q101. 描述 Python 的垃圾回收分代策略。

新对象在第 0 代，每代超过阈值（`gc.get_threshold()`）触发回收，存活对象提升到下一代；老一代回收频率低。3.12/3.13 对 GC 触发策略与增量回收做过调整，具体行为以版本说明为准。

Q102. 如何实现一个 LRU 缓存？

使用 `collections.OrderedDict`：

```
from collections import OrderedDict
class LRU:
    def __init__(self, cap): self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d: return -1
        self.d.move_to_end(k); return self.d[k]
    def put(self, k, v):
        self.d[k] = v; self.d.move_to_end(k)
        if len(self.d) > self.cap: self.d.popitem(last=False)
```

Q103. 如何实现一个线程安全的计数器？

```
import threading
class Counter:
    def __init__(self): self._n = 0; self._lock = threading.Lock()
    def inc(self):
        with self._lock: self._n += 1
```

`+=` 不是原子操作，必须加锁；多进程用 `multiprocessing.Value` 并配合锁。

Q104. 如何用生成器处理大数据管道？

把读取、转换、过滤、聚合写成串联的生成器函数，每次只处理一个元素，内存占用恒定：`lines = (l.strip() for l in f)` → `parsed = (parse(l) for l in lines)` → `sum(1 for r in parsed if r.ok)`。

Q105. 在 Python 中如何做深度比较、排序自定义对象？

为类实现 `__eq__`、`__lt__`（再用 `functools.total_ordering` 补全），或传入 `key=attrgetter('a', 'b')`；dataclass 可 `order=True`。

Q106. 如何统计一个列表中出现次数最多的元素？

`Counter(lst).most_common(1)`；或手写字典计数，时间复杂度 O(n)。

Q107. 回文、两数之和、反转链表等算法题在 Python 中的惯用写法？

回文：`s == s[::-1]`；两数之和：字典存值到下标；反转链表：迭代三指针。面试需说明时间/空间复杂度，注意边界（空输入、重复值、整数范围）。

Q108. Python 3.14 之后值得关注的趋势？

free-threaded 构建的成熟度、JIT 的持续改进、更快的启动与解释器、类型系统（新泛型语法、延迟注解）、`uv` / `ruff` 等 Rust 工具链的普及。生产使用前请确认依赖兼容性。

第五部分：Go（Golang）

5.1 语言基础

Q1. Go 语言有哪些特点？

静态强类型、编译为本地二进制、自带垃圾回收、内置并发原语（goroutine、channel）、结构化的接口（隐式实现）、快速编译、标准库丰富、工具链统一（`go build/test/vet/fmt/mod`）。不提供继承、异常、方法重载；1.18 起支持泛型。

Q2. `var`、`:=`、`const` 的区别？

`var x int` 可在包级和函数内使用，零值初始化；`x := 1` 短变量声明只能在函数内，并且至少有一个新变量（否则编译报错，注意与外层变量遮蔽 shadowing 的问题）；`const` 为编译期常量，可以是无类型常量（有任意精度）。

Q3. iota 如何使用？

在 const 块中表示从 0 开始的行号，可用于枚举：

```
const (
    KB = 1 << (10 * (iota + 1))
    MB
    GB
)
```

省略表达式时沿用上一行的表达式。

Q4. 各类型的零值是什么？

数值 0、bool false、string ""、指针/slice/map/chan/func/interface 为 nil、结构体字段逐个零值。"让零值可用"是 Go 的设计习惯，如 `sync.Mutex`、`bytes.Buffer` 零值即可用。

Q5. 数组和切片的区别？

数组长度是类型的一部分（`[3]int` 与 `[4]int` 不同类型），是值类型，赋值/传参会整体拷贝；切片是对底层数组的引用视图，包含指针、长度、容量三个字段（24 字节），传参时拷贝的是切片头。

Q6. slice 的扩容规则？

`append` 容量不足时分配新数组并拷贝。Go 1.18+：小于 256 个元素时约翻倍，之后逐步平滑过渡到约 1.25 倍（公式 `newcap += (newcap + 3*256) / 4`），最后还要按内存大小类（size class）向上取整，所以实际容量可能比公式大。不要依赖具体数值。

Q7. 切片共享底层数组会有什么坑？

```
a := []int{1,2,3,4}
b := a[:2]
b = append(b, 99)   // 容量足够，会覆盖 a[2]
```

子切片的 append 可能修改原数组；可用完整切片表达式 `a[:2:2]` 限制容量迫使 append 重新分配。大数组的小切片会阻止整个底层数组被回收，需要时 `copy` 出来。

Q8. nil 切片与空切片有什么区别？

`var s []int` 为 nil（`s == nil`），`[]int{}` 非 nil 但长度 0。二者 `len`、`cap`、`append`、`range` 行为一致；JSON 序列化时 nil 切片为 `null`，空切片为 `[]`。

Q9. map 的底层实现？

Go 1.24 前是哈希表：`hmap` + 若干桶（每桶 8 个键值对），溢出桶链接，负载因子约 6.5，增量式扩容（渐进搬迁）。Go 1.24 起默认改用 Swiss Table 实现。对使用者的语义不变：无序、遍历顺序随机、非并发安全。

Q10. map 可以并发读写吗？

并发读可以；有写操作时并发读写会触发运行时 `fatal error: concurrent map read and map write`（不可 recover）。解决：`sync.RWMutex` 保护，或

`sync.Map`（读多写少、键集合稳定的场景），或分片锁。

Q11. map 的键有什么要求？

键类型必须可比较（支持 `==`）：基础类型、指针、channel、interface、数组、只含可比较字段的结构体。slice、map、func 不能作键。NaN 作键时每次插入都是新键。

Q12. 对 map 遍历顺序、delete、nil map 有什么需要注意？

遍历顺序故意随机化；遍历时删除是安全的，遍历时新增可能出现也可能不出现；读 nil map 返回零值，写 nil map 会 panic；`delete` 对不存在的键无操作。

Q13. string 的底层与遍历？

string 是不可变的字节序列（指针+长度），`len(s)` 是字节数；`for i, r := range s` 按 UTF-8 解码得到 rune；`[]rune(s)` 转换用于按字符处理；`s[i]` 是字节。大量拼接使用 `strings.Builder`。

Q14. byte 与 rune 的区别？

`byte` 是 `uint8` 的别名，`rune` 是 `int32` 的别名，表示 Unicode 码点。

Q15. new 和 make 的区别？

`new(T)` 分配零值内存并返回 `*T`；`make` 只用于 slice、map、channel，返回初始化后的值（非指针），如 `make([]int, 0, 10)`、`make(map[string]int, hint)`、`make(chan int, 5)`。

Q16. 结构体可以比较吗？

当所有字段都可比较时可用 `==`；包含 slice、map、func 字段则不可比较。可比较的结构体可以做 map 键。

Q17. 结构体内存对齐如何优化？

字段按自身对齐要求排列，整体按最大对齐补齐。把大字段放前面（或按大小降序排列）可减少填充；`fieldalignment` 分析器可检测。`struct{}` 大小为 0，常用于 `map[T]struct{}` 做集合和 `chan struct{}` 做信号。

Q18. 结构体标签（tag）有什么用？

通过反射读取，用于 JSON、数据库、校验等：`json:"name,omitempty"`、`db:"id"`、`validate:"required"`。

Q19. Go 1.21 到 1.25 的重要变化有哪些？

1.18 泛型、fuzzing、workspace；1.19 `GOMEMLIMIT`、`atomic` 类型；1.20 `errors.Join`；1.21 内置 `min` / `max` / `clear`、`log/slog`、`slices` / `maps` 包、PGO 正式；1.22 for 循环变量每次迭代独立、`range` 整数、`net/http.ServeMux` 增强的模式匹配；1.23 range-over-func 迭代器与 `iter` 包、timer 行为修复；1.24 Swiss map、泛型类型别名、`testing.B.Loop`、`weak` 包；1.25 `sync.WaitGroup.Go`、容器感知的 `GOMAXPROCS`。具体以官方发布说明为准。

5.2 函数、错误与 defer

Q20. defer 的执行顺序和参数求值时机？

后进先出（LIFO）；`defer` 语句执行时就会对参数求值，而函数体在返回前才执行：

```
for i := 0; i < 3; i++ { defer fmt.Println(i) }   // 输出 2 1 0
```

用闭包 `defer func(){ ... }()` 可延迟读取变量最新值。

Q21. defer 如何与命名返回值交互？

`return x` 先给返回值赋值，再执行 defer，最后返回。defer 里的闭包可以修改命名返回值：

```
func f() (n int) { defer func(){ n++ }(); return 1 }   // 返回 2
```

Q22. panic 和 recover 的使用规则？

`panic` 触发后逐层执行 defer，若无人 recover 则程序崩溃。`recover` 只在 defer 直接调用的函数里有效，且只能恢复本 goroutine 的 panic。典型用

法：服务器框架在顶层 recover 防止单个请求崩溃整个进程。不要用 panic 做常规错误处理。

Q23. Go 的错误处理为什么是返回 error？

显式、简单：`error` 是接口 `Error() string`，调用者必须决定怎么处理。缺点是样板代码多。包装错误：`fmt.Errorf("read config: %w", err)`；判断：`errors.Is(err, target)`、`errors.As(err, &target)`；合并多个：`errors.Join`。

Q24. 哨兵错误、自定义错误类型、错误包装如何选择？

需要比较特定错误值用哨兵（`io.EOF`）；需要携带额外信息用自定义类型（实现 `Error()`，配合 `errors.As`）；跨层传递加上下文用 `%w` 包装。注意不要把 `nil` 指针的自定义错误类型当作 error 返回（见 Q34）。

Q25. 闭包捕获循环变量的问题？

Go 1.22 之前，`for i := ...` 的变量在整个循环中是同一个，goroutine 或 defer 中捕获会看到最后的值。Go 1.22 起每次迭代都有独立变量（以 go.mod 中的 `go` 版本为准）。旧版本写法：`i := i`。

Q26. 可变参数函数如何使用？

`func sum(nums ...int)`，调用 `sum(a...)` 传切片（不拷贝，共享底层数组）。可变参数必须是最后一个参数。

Q27. 函数是一等公民意味着什么？

函数可赋值给变量、作为参数/返回值、构成闭包；方法值 `f := obj.Method` 会绑定接收者（按值接收者时拷贝一份）；方法表达式 `T.Method` 需显式传入接收者。

Q28. init 函数与包初始化顺序？

先初始化依赖包，再按依赖关系初始化包级变量，最后执行 `init()`（一个包可有多个，按文件名顺序提交给编译器的顺序执行）。不要在 `init` 中做复杂副作用，影响可测试性。

Q29. 值接收者和指针接收者怎么选？

需要修改接收者、结构体较大、包含 `sync.Mutex` 等不可拷贝字段、或者要保持一致性时用指针接收者；小的不可变值类型（如 `time.Time`）用值接收者。同一类型的方法最好统一风格。

Q30. 方法集规则是什么？

类型 `T` 的方法集只包含值接收者方法，`*T` 的方法集包含值和指针接收者方法。因此 `*T` 能满足接口，而 `T` 若有指针接收者方法则不能。可寻址变量调用指针方法时编译器自动取地址，但 map 元素不可寻址。

5.3 接口与类型系统

Q31. Go 接口的特点？

隐式实现（结构化类型）：只要实现了接口的所有方法即满足，无需声明。小接口哲学（`io.Reader`、`io.Writer`）；"接受接口，返回结构体"；接口应由使用方定义。

Q32. 空接口与 any 是什么？

`interface{}` 可存任意值，Go 1.18 起 `any` 是其别名。使用类型断言 `v, ok := x.(T)` 或 `switch x.(type)` 取出具体类型。泛型出现后应优先使用泛型而不是 `any`。

Q33. 接口的底层结构是什么？

非空接口为 `iface{tab *itab, data unsafe.Pointer}`，`itab` 包含接口类型、动态类型和方法表；空接口为 `eface{_type, data}`。接口赋值可能触发堆分配（值需要装箱）。

Q34. 为什么会出现"接口不为 nil"的坑？

接口只有在动态类型和值都为 nil 时才等于 nil：

```
var p *MyErr = nil
var err error = p
fmt.Println(err == nil)   // false
```

函数返回 error 时，应直接 `return nil`，不要返回类型化的 nil 指针。

Q35. 类型断言和类型开关怎么写？

```
if s, ok := v.(fmt.Stringer); ok { ... }
switch t := v.(type) {
case int:    ...
case string: ...
default:     ...
}
```

不带 ok 的断言失败会 panic。

Q36. Go 如何实现"继承"？结构体嵌入是什么？

通过组合：在结构体中嵌入类型，其字段和方法被提升（promoted），外层可直接调用。这不是继承——没有多态（嵌入类型调用方法时接收者仍是内部类型），需要多态用接口。字段冲突时需要显式限定。

Q37. 接口值如何比较？

动态类型相同且值相等时才相等；如果动态类型不可比较（slice、map），比较会 panic。

Q38. 泛型怎么写？约束是什么？

```
func Map[T, U any](s []T, f func(T) U) []U { ... }
type Number interface{ ~int | ~int64 | ~float64 }
func Sum[T Number](xs []T) (s T) { for _, x := range xs { s += x }; re
turn }
```

`comparable` 约束允许 `==`；`~int` 表示底层类型为 int 的所有类型。标准库 `cmp.Ordered`、`slices`、`maps` 提供常用实现。

Q39. 泛型的实现机制与限制？

基于"GC shape stenciling + 字典"：同一内存形状的类型共享实例化代码，用字典传递类型信息，权衡了编译体积与性能。限制：方法不能有自己的类型参数；不能特化；无泛型的方法重载。

Q40. 什么时候不应该使用泛型？

只有一种具体类型、用接口即可表达行为、或者泛型使代码更难读时。常见合理使用：容器、通用算法（Map/Filter/Reduce）、数据结构（栈、队列）。

5.4 并发

Q41. goroutine 与线程的区别？

goroutine 是用户态轻量协程，初始栈约 2KB（可动态增长到默认 1GB 上限），创建和切换成本极低，由 Go 运行时调度到少量 OS 线程上；线程栈固定几 MB，由内核调度。可轻松创建数十万 goroutine。

Q42. GMP 调度模型是什么？

G（goroutine）、M（OS 线程）、P（逻辑处理器，持有本地运行队列，数量为 `GOMAXPROCS`）。M 必须绑定 P 才能运行 G。调度策略：本地队列（长度 256）→ 全局队列 → 网络轮询器 → 从其他 P 窃取（work stealing）；系统调用阻塞时 P 会与 M 解绑交给其他 M；sysmon 监控线程负责抢占长时间运行的 G。

Q43. Go 的抢占调度是怎样演进的？

1.2 起在函数调用处检查抢占（协作式）；1.14 引入基于信号的异步抢占，解决了紧凑循环无法被抢占的问题。

Q44. channel 的基本特性？

`make(chan T)` 无缓冲：发送与接收同步（rendezvous）；`make(chan T, n)` 有缓冲：缓冲满时发送阻塞、空时接收阻塞。可通过 `chan<-`、`<-chan` 限定方向。`for v := range ch` 直到 channel 关闭。

Q45. channel 在 nil、已关闭状态下的行为？

- nil channel：发送、接收永久阻塞（在 select 中可用于禁用某个分支）。

- 已关闭：接收立即返回零值且 `ok == false`（缓冲内的数据先读完）；发送会 panic；再次 close 会 panic；close nil channel 会 panic。 惯例：由发送方关闭，不要由接收方关闭；多发送者时用额外信号（context、done channel）协调。

Q46. channel 的底层实现？

`hchan` 结构：环形缓冲区、`sendx/recvx`、等待队列 `sendq/recvq`（`sudog` 链表）、互斥锁。发送时若有等待的接收者则直接拷贝给它；否则入缓冲或阻塞挂起当前 G。

Q47. select 的行为？

同时等待多个 channel 操作：多个就绪时随机选择一个；都不就绪时有 `default` 则执行，否则阻塞。常用于超时（`time.After` / `context`）、取消、多路复用。空 `select {}` 永久阻塞。

Q48. sync.Mutex 与 RWMutex 的要点？

Mutex 不可重入、不可拷贝（`go vet` 的 copylocks 会检查）；有正常模式与饥饿模式（等待超过 1ms 切换为饥饿模式，防止长期等待）。
RWMutex 允许多读单写，但读多写少时才划算，并注意写锁等待时会阻止新读者。

Q49. WaitGroup、Once、Cond、Pool 的用途？

`WaitGroup` 等待一组 goroutine（`Add` 要在启动 goroutine 之前调用；Go 1.25 有 `wg.Go`）；`Once` 保证只执行一次（`sync.OnceFunc/OnceValue`，1.21）；`Cond` 条件变量；`Pool` 复用临时对象减轻 GC 压力（对象可能随时被回收，不可存放有状态连接）。

Q50. sync.Map 的适用场景？

适合两种场景：键写入一次读取多次；多个 goroutine 操作互不重叠的键集合。其他场景通常用 `map` + `RWMutex` 更简单更快。

Q51. atomic 包如何使用？

`atomic.AddInt64`、`CompareAndSwap`，以及 1.19 起的类型 `atomic.Int64`、`atomic.Bool`、`atomic.Pointer[T]`、`atomic.Value`。注意 64 位原子变量在 32 位平台上的对齐要求，使用新类型可避免。

Q52. context 包的作用与规范？

在调用链中传递取消信号、截止时间和请求范围的值：`context.WithCancel/WithTimeout/WithDeadline/WithValue`，`ctx.Done()` 和 `ctx.Err()`。规范：作为函数第一个参数、命名为 `ctx`；不要存入结构体；`cancel` 必须调用（`defer cancel()`）；`WithValue` 只放请求范围元数据，不放可选参数。

Q53. 什么是 goroutine 泄漏？如何避免？

goroutine 因永久阻塞（无人接收的 channel、没有退出条件的循环、未取消的 context）而无法结束，导致内存增长。避免：为每个 goroutine 设计退出路径（context/done channel）、有缓冲 channel 防止发送阻塞、使用 `errgroup`；排查：`pprof` 的 goroutine profile、`runtime.NumGoroutine`。

Q54. 如何检测数据竞争？

`go test -race`、`go run -race`，基于 ThreadSanitizer，有 5–10 倍的运行开销，适合测试和灰度环境。数据竞争在 Go 内存模型中是未定义的（可能得到任意值）。

Q55. Go 内存模型（happens-before）的要点？

保证 data-race-free 的程序行为如同顺序一致执行。同步事件：channel 发送 happens-before 对应接收完成；关闭 channel happens-before 接收到关闭；`Unlock` happens-before 下一次 `Lock`；`Once.Do` 中函数返回 happens-before 任何 `Do` 返回；atomic 操作表现为顺序一致。

Q56. 如何实现 worker pool？

```
jobs := make(chan Job)
var wg sync.WaitGroup
for i := 0; i < n; i++ {
    wg.Add(1)
    go func() { defer wg.Done(); for j := range jobs { handle(j) } }()
}
for _, j := range all { jobs <- j }
close(jobs); wg.Wait()
```

需要错误传播与取消时用 `golang.org/x/sync/errgroup`（`SetLimit` 限制并发）。

Q57. 如何用 channel 实现信号量或限流？

带缓冲的 channel 作为令牌：`sem := make(chan struct{}, N)`，执行前 `sem <- struct{}{}`，结束后 `<-sem`。更完善的限流用 `golang.org/x/time/rate`（令牌桶）或 `x/sync/semaphore`。

Q58. fan-in / fan-out 与 pipeline 模式？

pipeline：多个阶段通过 channel 串联；fan-out：多个 goroutine 从同一 channel 消费以并行；fan-in：多个 channel 汇聚到一个。每个阶段负责关闭自己的输出 channel，并响应 context 取消。

Q59. time.After 在循环里有什么坑？

Go 1.23 之前，未触发的 `time.After` 创建的 Timer 在到期前不会被 GC 回收，循环中大量使用会造成泄漏；1.23 起未引用的 timer 可被回收。仍建议在高频循环中使用 `time.NewTimer` 并 `Stop/Reset`。`Ticker` 使用后必须 `Stop`。

Q60. 如何优雅地停止 goroutine？

通过 `context` 取消或关闭 `done` channel；让阻塞操作用 `select` 同时监听退出信号；避免在 goroutine 内直接 `os.Exit`。`signal.NotifyContext(ctx, os.Interrupt, syscall.SIGTERM)` 用于进程级信号。

Q61. 一个 goroutine panic 会怎样？

未被该 goroutine 自己 recover 的 panic 会使整个进程崩溃（无法在其他 goroutine 中捕获）。启动 goroutine 的封装函数里应加 recover 并记录。

Q62. 无缓冲与有缓冲 channel 如何取舍？

无缓冲提供强同步语义，容易暴露死锁；有缓冲可以解耦生产者与消费者、削峰，但容量要有依据，不要把缓冲当作"修复死锁"的手段。

Q63. 死锁（all goroutines are asleep）是怎么产生的？

所有 goroutine 都在等待（无缓冲 channel 没有对端、`wg.Wait` 的计数永不归零、锁嵌套等），运行时检测到会抛出 `fatal error: all goroutines are asleep - deadlock!`。仅当所有 goroutine 都阻塞时才会检测到，部分死锁不会报错。

Q64. sync.Mutex 与 channel 如何选择？

"通过通信共享内存"适合所有权转移、流水线、事件通知；互斥锁适合保护共享状态的短临界区（缓存、计数）。并没有绝对优劣，选择更简单直观的那个。

Q65. 如何实现带超时的调用？

```
ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
defer cancel()
select {
case r := <-doWork(ctx): ...
case <-ctx.Done(): return ctx.Err()
}
```

被调用方也必须检查 `ctx.Done()`，否则超时后 goroutine 仍在后台运行。

Q66. Go 1.23 的迭代器（range-over-func）是什么？

```
func Seq(n int) iter.Seq[int] { return func(yield func(int) bool) { fo
r i := 0; i < n; i++ { if !yield(i) { return } } } }
for v := range Seq(5) { ... }
```

`iter.Seq` / `iter.Seq2` 配合 `slices.Collect`、`maps.Keys` 等；`iter.Pull` 可转为拉取式。

5.5 内存与运行时

Q67. Go 的垃圾回收算法？

并发三色标记-清除，配合混合写屏障（Go 1.8+）；STW 阶段极短（通常亚毫秒）；非分代、非整理（不移动对象）。GC 触发由 `GOGC`（默认 100，即堆增长 100% 时触发）与 `GOMEMLIMIT`（1.19 起的软内存上限）控制。

Q68. 什么是逃逸分析？如何查看？

编译器决定变量分配在栈还是堆：被返回的指针、闭包捕获、接口装箱、过大对象、大小在编译期未知等会逃逸到堆。查看：`go build -gcflags='-m'`（`-m -m` 更详细）。减少逃逸可降低 GC 压力。

Q69. goroutine 栈是如何增长的？

初始约 2KB，函数序言检查栈空间不足时，分配更大（通常 2 倍）的连续栈并拷贝旧栈内容，更新栈内指针。栈也可在 GC 时缩小。因此不要依赖栈上变量的地址保持不变。

Q70. Go 的内存分配器如何设计？

类似 TCMalloc：按大小类（size class）分级；每个 P 有本地缓存 `mcache`（无锁），不足时向 `mcentral` 要 span，再向 `mheap` 申请页；大对象（>32KB）直接从 `mheap` 分配。小对象无指针的使用 noscan span，GC 不必扫描。

Q71. 如何调优 GC？

先用 `pprof`/trace 确认是否是瓶颈；减少分配（复用对象、预分配切片/map 容量、`sync.Pool`、避免不必要的装箱）；适当调高 `GOGC` 用内存换 CPU；容器环境设置 `GOMEMLIMIT` 防止 OOM；避免大量小对象的指针（降低标记成本）。

Q72. 如何做性能分析（pprof 与 trace）？

引入 `net/http/pprof` 或 `runtime/pprof`；`go tool pprof` 查看 CPU、heap（inuse/alloc）、goroutine、block、mutex profile；`go tool trace` 查看调度、GC、阻塞事件；基准测试 `go test -bench . -benchmem -cpuprofile`。生产环境可使用持续剖析（Pyroscope、Parca）。

Q73. 为什么 Go 里内存泄漏仍然会发生？

全局 map/缓存无界增长、goroutine 泄漏、`time.Ticker` 未停止、切片截取持有大数组、`defer` 在循环中累积、`http.Response.Body` 未关闭。

Q74. unsafe 包能做什么？有何风险？

`unsafe.Pointer` 可在任意指针类型间转换，`unsafe.Sizeof/Offsetof/Alignof`，`unsafe.String/Slice/StringData`（1.20）用于零拷贝转换。风险：绕过类型安全和 GC 规则，可能造成崩溃或内存损坏，只应在性能关键且严格遵守文档规则的场合使用。

Q75. cgo 有什么代价？

Go 与 C 调用有额外开销（约几十到上百纳秒量级），C 代码不受 GC 管理，阻塞调用会占用线程，交叉编译困难、二进制不再是纯静态。能用纯 Go 就不用 cgo；`CGO_ENABLED=0` 可得到静态二进制。

Q76. 反射的使用与代价？

`reflect` 可在运行期检查和操作类型/值，是 `encoding/json`、ORM 等的基础。代价：慢、类型不安全、难以静态分析。泛型和代码生成可替代部分场景。

Q77. 值传递与引用语义在 Go 中怎么理解？

Go 只有值传递，传 slice/map/chan/func/指针时拷贝的是头部或指针，所以看起来像"引用语义"。修改 slice 元素会影响原数据，但 `append` 后的新长度不会反映到调用者的切片头。

5.6 工具链与工程实践

Q78. go modules 如何工作？

`go.mod` 声明模块路径、go 版本、依赖（语义化版本，`v2+` 的路径带 `/v2`）；`go.sum` 记录哈希校验；使用最小版本选择（MVS）。常用命令：`go mod init/tidy/download/vendor`、`replace` 本地调试、`go.work` 多模块工作区、`toolchain` 指令（1.21）自动选择工具链、`GOPROXY`/`GOPRIVATE` 配置私有仓库。

Q79. 包与可见性规则？

标识符首字母大写即导出；包名小写、简短、不用下划线；`internal/` 目录限制只能被父目录下的代码导入；避免循环导入（可抽出接口或公共包）。

Q80. 如何写测试？table-driven 测试是什么？

文件名 `_test.go`，函数 `TestXxx(t *testing.T)`。表驱动：

```
          tests := []struct{ name string; in, want int }{ {"zero", 0, 0}, {"on
e", 1, 1} }
          for _, tt := range tests {
              t.Run(tt.name, func(t *testing.T) {
                  if got := F(tt.in); got != tt.want { t.Errorf("F(%d)=%d want %
d", tt.in, got, tt.want) }
              })
          }
```

子测试可 `t.Parallel()`；`testify` 提供断言和 mock，`httptest` 测试 HTTP，`testing/synctest`（1.25）测试并发时间逻辑。

Q81. 基准测试与模糊测试怎么写？

基准：`func BenchmarkX(b *testing.B) { for i := 0; i < b.N; i++ { ... } }`（1.24 起可用 `for b.Loop()`），运行 `go test -bench . -benchmem`。模糊测试（1.18）：`func FuzzX(f *testing.F)`，`go test -fuzz=FuzzX`，自动发现边界输入。

Q82. go vet、gofmt、staticcheck、golangci-lint 的作用？

`gofmt/goimports` 统一格式；`go vet` 检查可疑构造（Printf 格式、copylocks、loopclosure 等）；`staticcheck`、`golangci-lint` 聚合多个 linter。CI 中应强制执行。

Q83. 如何交叉编译与减小二进制体积？

`GOOS=linux GOARCH=amd64 go build`；`CGO_ENABLED=0` 获得静态二进制；`-ldflags "-s -w"` 去除符号和调试信息；`-trimpath`；通过 `-ldflags "-X main.version=..."` 注入版本信息；容器镜像可用 distroless/scratch。

Q84. 如何用 net/http 写一个健壮的服务？

设置 `http.Server` 的 `ReadHeaderTimeout/ReadTimeout/WriteTimeout/IdleTimeout`，使用中间件（日志、recover、鉴权、限流、trace），Go 1.22 起 `ServeMux` 支持 `"GET /users/{id}"` 模式，优雅关闭用 `server.Shutdown(ctx)`。客户端复用 `http.Client`（连接池）、设置超时、关闭 `resp.Body`。

Q85. 如何实现优雅关闭？

```
          ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
          defer stop()
          go srv.ListenAndServe()
          <-ctx.Done()
          shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*ti
me.Second)
          defer cancel()
          srv.Shutdown(shutdownCtx)
```

同时通知后台 worker 退出并等待，关闭数据库/消息连接。

Q86. encoding/json 的常见问题？

未导出字段不会序列化；`omitempty` 对零值的处理（如 0、false、空串）；`json.RawMessage` 延迟解析；数字解到 `interface{}` 默认是 `float64`

（大整数丢精度，可用 `Decoder.UseNumber`）；流式大数据用 `Decoder`；未知字段可用 `DisallowUnknownFields` 拒绝。

Q87. database/sql 连接池如何配置？

`SetMaxOpenConns`、`SetMaxIdleConns`、`SetConnMaxLifetime`、`SetConnMaxIdleTime`。常见问题：未 `rows.Close()`、`Tx` 未提交/回滚造成连接泄漏、`context` 未传入导致慢查询无法取消、预编译语句滥用。

Q88. log/slog 结构化日志如何使用？

Go 1.21 标准库：`slog.Info("req", "method", m, "status", s)`，`slog.With(...)` 附加字段，`JSONHandler` 输出 JSON，可自定义 Handler 与级别。在 HTTP 中结合 context 传递 trace ID。

Q89. 常见的项目目录结构？

没有官方强制规范；常见 `cmd/<app>/main.go`、`internal/`（私有包）、`pkg/`（对外库，社区有争议）、`api/`、`configs/`。小项目保持扁平，按业务领域而不是技术层划分包。

Q90. 依赖注入在 Go 中怎么做？

首选手动构造函数注入（显式、简单）；大型项目可用编译期生成的 `google/wire` 或运行期的 `uber-go/fx` / `dig`。面向接口编程便于 mock。

Q91. Go 的 gRPC 与 Protobuf 使用要点？

`.proto` 定义服务，`protoc-gen-go` / `protoc-gen-go-grpc`（或 buf）生成代码；使用拦截器做日志/鉴权/重试，传递 deadline；注意向后兼容（不重用字段号）、连接复用与健康检查、流式 RPC 的取消。

Q92. 如何设计可测试的 Go 代码？

依赖通过接口注入，时间/随机数/IO 抽象为接口，避免全局状态与 `init` 副作用，小函数、纯函数优先，使用 `httptest`、临时目录 `t.TempDir()`、`t.Cleanup`。

Q93. Go 里的常见代码风格要点？

`gofmt` 统一格式；错误字符串小写不加标点；接口命名常以 `-er` 结尾；避免包名重复（`user.UserService` → `user.Service`）；尽早 return，减少嵌套；导出标识符写文档注释；函数参数 `ctx` 在前；避免滥用 `init` 和全局变量。

Q94. Go 与 Java、Python 在服务端开发上有什么差异？

Go：静态编译部署简单、启动快、内存占用低、并发模型简单，缺少成熟的重型框架生态和某些语言特性（早期无泛型、错误处理冗长）。Java：生态庞大、JIT 峰值性能强但启动与内存开销较大。Python：开发效率高、科学计算与 AI 生态强，性能和并发受 GIL 限制。

Q95. 如何排查线上 Go 服务 CPU 或内存飙高？

开启 pprof 端点抓取 profile（`/debug/pprof/profile?seconds=30`、`heap`、`goroutine?debug=2`），用 `go tool pprof -http` 看火焰图；结合 `runtime.MemStats`、GC trace（`GODEBUG=gctrace=1`）和 `go tool trace`；留意 goroutine 数量异常增长和锁竞争（`mutex` / `block` profile）。

Q96. 常见的限流、熔断、重试在 Go 中的实现？

限流：`x/time/rate`；熔断：`sony/gobreaker`；重试：指数退避 + 抖动 + 上限，并确保幂等；超时：所有出站调用都设置 context deadline。

Q97. 在 Go 中如何实现一个线程安全的 LRU 缓存？

`container/list` 双向链表 + `map[key]*list.Element` + `sync.Mutex`：

```
type LRU struct{ mu sync.Mutex; cap int; ll *list.List; m map[string]*
list.Element }
type entry struct{ k string; v int }
func (c *LRU) Get(k string) (int, bool) {
    c.mu.Lock(); defer c.mu.Unlock()
    if e, ok := c.m[k]; ok { c.ll.MoveToFront(e); return e.Value.(*ent
ry).v, true }
    return 0, false
}
func (c *LRU) Put(k string, v int) {
    c.mu.Lock(); defer c.mu.Unlock()
    if e, ok := c.m[k]; ok { e.Value.(*entry).v = v; c.ll.MoveToFront
(e); return }
    c.m[k] = c.ll.PushFront(&entry{k, v})
    if c.ll.Len() > c.cap { b := c.ll.Back(); c.ll.Remove(b); delete
(c.m, b.Value.(*entry).k) }
}
```

Q98. 如何实现并发安全的单例？

`sync.Once`：

```
var (inst *T; once sync.Once)
func Get() *T { once.Do(func(){ inst = &T{} }); return inst }
```

或 `sync.OnceValue`（1.21）。包级变量初始化也是线程安全的。

Q99. 标准库里值得掌握的包有哪些？

`fmt`、`io` / `bufio`、`os`、`strings` / `bytes` / `strconv`、`sort` / `slices` / `maps`、`errors`、`context`、`sync` / `sync/atomic`、`time`、`net/http`、`encoding/json`、`database/sql`、`log/slog`、`testing`、`regexp`、`flag`、`embed`（嵌入静态文件，1.16）。

Q100. embed 与 build tags 有什么用？

`//go:embed assets/*` 把文件编进二进制；`//go:build linux && amd64` 控制文件按条件编译，文件名后缀 `_linux.go`、`_test.go` 也有同样效果。

Q101. 什么是 PGO？

Profile-Guided Optimization（1.21 正式）：用 CPU profile（放在 `default.pgo`）指导内联和去虚化等优化，通常带来 2%~14% 的提升。

Q102. defer 的性能代价？

1.14 起引入 open-coded defer，大多数情况下开销接近普通函数调用；在循环中 defer 会累积直到函数返回，应改为在循环体内调用小函数。

Q103. 为什么 range 遍历 slice 修改元素不生效？

`for _, v := range s` 中的 `v` 是元素副本，需要用下标 `s[i] = ...` 或元素是指针。range 会在循环开始时对 slice 头求值一次，循环内追加元素不会延长遍历。

Q104. 数组的 range 和切片有何区别？

对数组 range 会拷贝整个数组（除非只用下标或遍历指针 `&arr`），对切片只拷贝切片头。

Q105. 字符串和字节切片转换的代价？

`[]byte(s)` 与 `string(b)` 会拷贝数据（编译器在部分场景可优化）。零拷贝可用 `unsafe.String` / `unsafe.Slice`，但要保证不修改和生命周期安全。

Q106. 如何对 map 按 key 排序遍历？

先收集 key 再排序：

```
keys := slices.Sorted(maps.Keys(m))   // Go 1.23
for _, k := range keys { use(m[k]) }
```

旧版本用 `sort.Strings(keys)`。

Q107. 类型别名与类型定义的区别？

`type A = B` 是别名，与 B 完全相同；`type A B` 是新类型，拥有 B 的底层类型但不继承方法。1.24 起支持泛型类型别名。

Q108. 方法值与闭包捕获的接收者拷贝问题？

`f := t.Method`（值接收者）会在求值时拷贝 `t`，之后修改 `t` 不影响 `f`；指针接收者则绑定指针。

Q109. 为什么 Go 没有异常机制？

设计者认为异常使控制流隐式、难以推理。Go 用显式错误返回和 `defer/panic/recover` 处理真正异常的情况。代价是样板代码，可通过辅助函数、错误包装来缓解。

Q110. 面试中常问的 Go 经典代码题？

交替打印奇偶数/字母数字（两个 goroutine + channel）、用 channel 实现生产者消费者、实现并发限流、实现 LRU、带超时的 HTTP 请求、并发安全的计数器、`sync.WaitGroup` 等待任务、判断 slice 扩容后的容量与地址。核心是说明同步方式、退出条件与资源释放。
